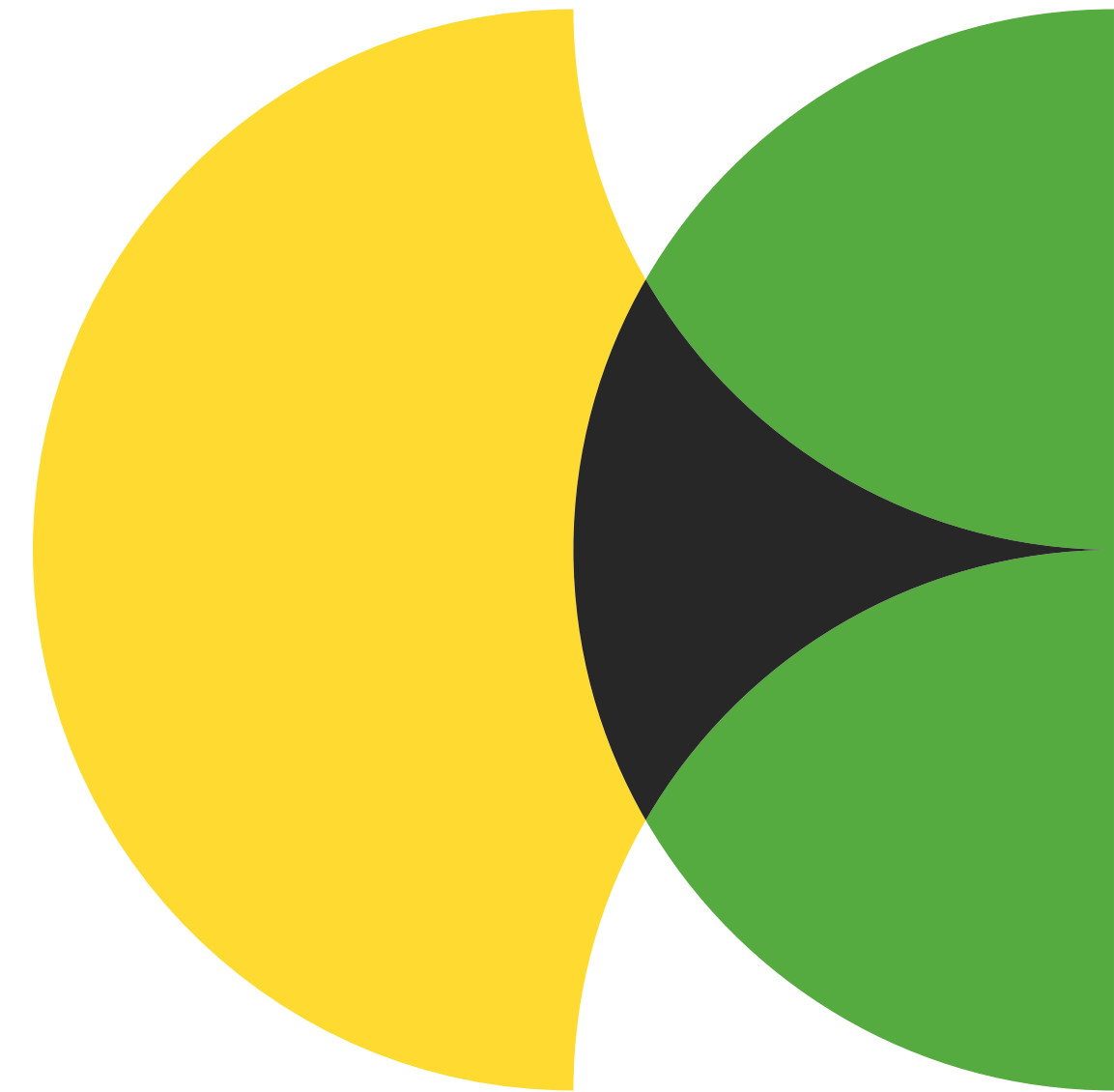




# Padrões de Projeto Comportamentais

Matheus R. Kirzner

Padrões comportamentais  
tratam da criação de  
algoritmos e atribuição de  
responsabilidade entre  
objetos.



# Índice

**1 - CHAIN OF RESPONSIBILITY**

**2 - COMMAND**

**3 - ITERATOR**

**4 - MEDIATOR**

**5 - MEMENTO**

**6 - OBSERVER**

**7 - STATE**

**8 - STRATEGY**

**9 - TEMPLATE METHOD**

**10 - VISITOR**



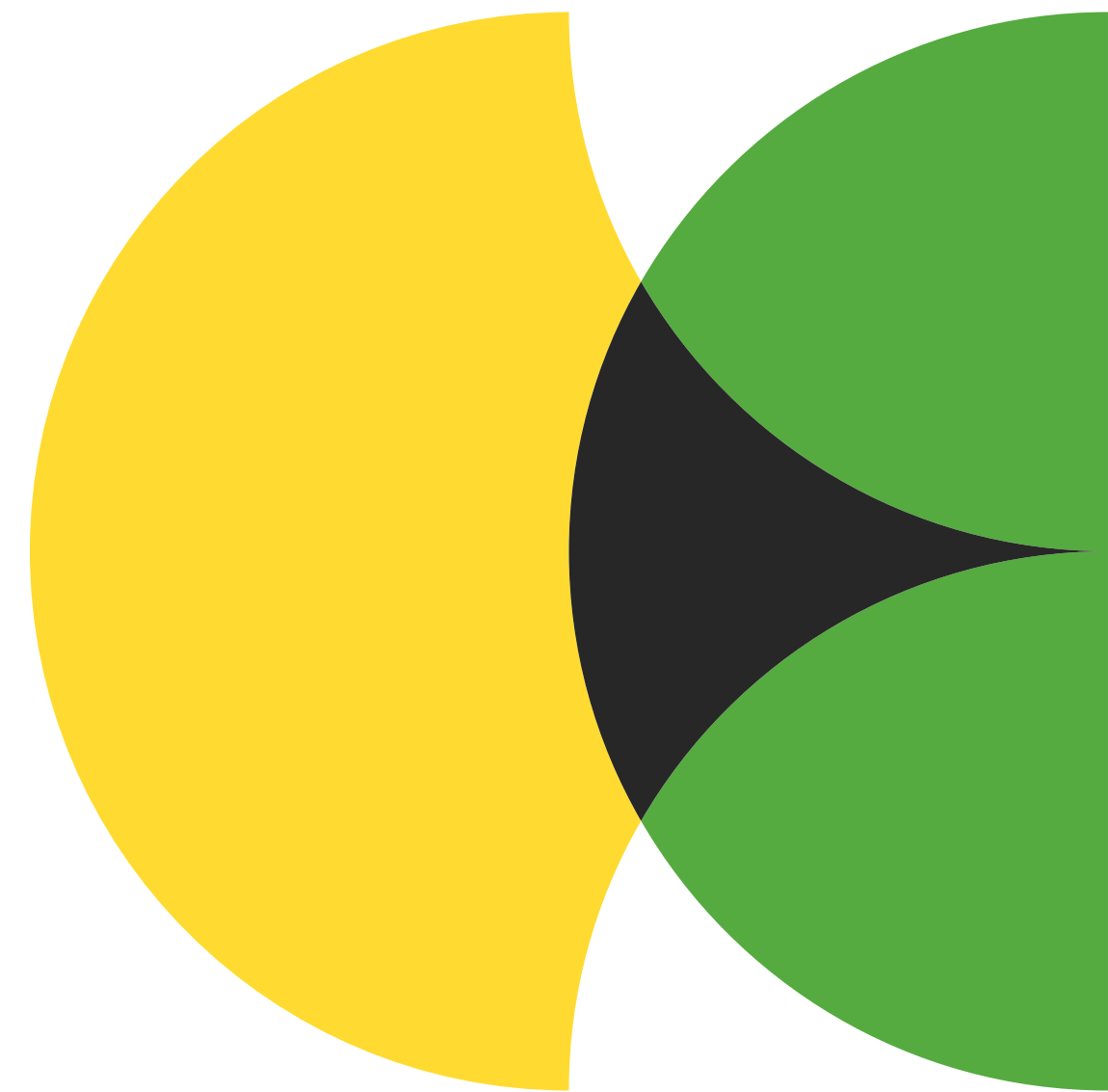


# Chain of Responsibility

Também conhecido como: CoR, Corrente de responsabilidade, Corrente de comando, Chain of command

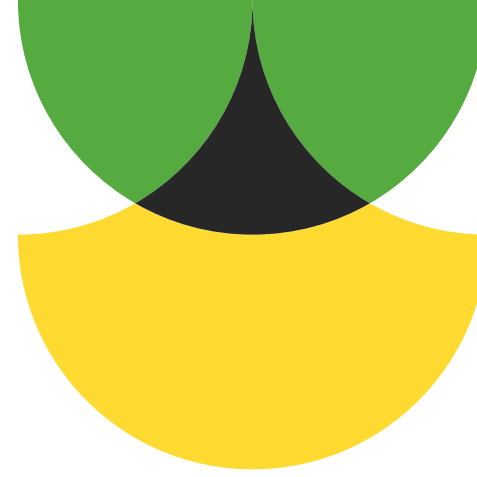


É o padrão de projeto que permite passar pedidos por uma corrente de handlers. Cada handler decide se resolve ou repassa o pedido.



# O problema:

Precisamos fazer uma sequência de verificações. Entretanto, o processo pode ser custoso ou de difícil manutenção.



# A solução:

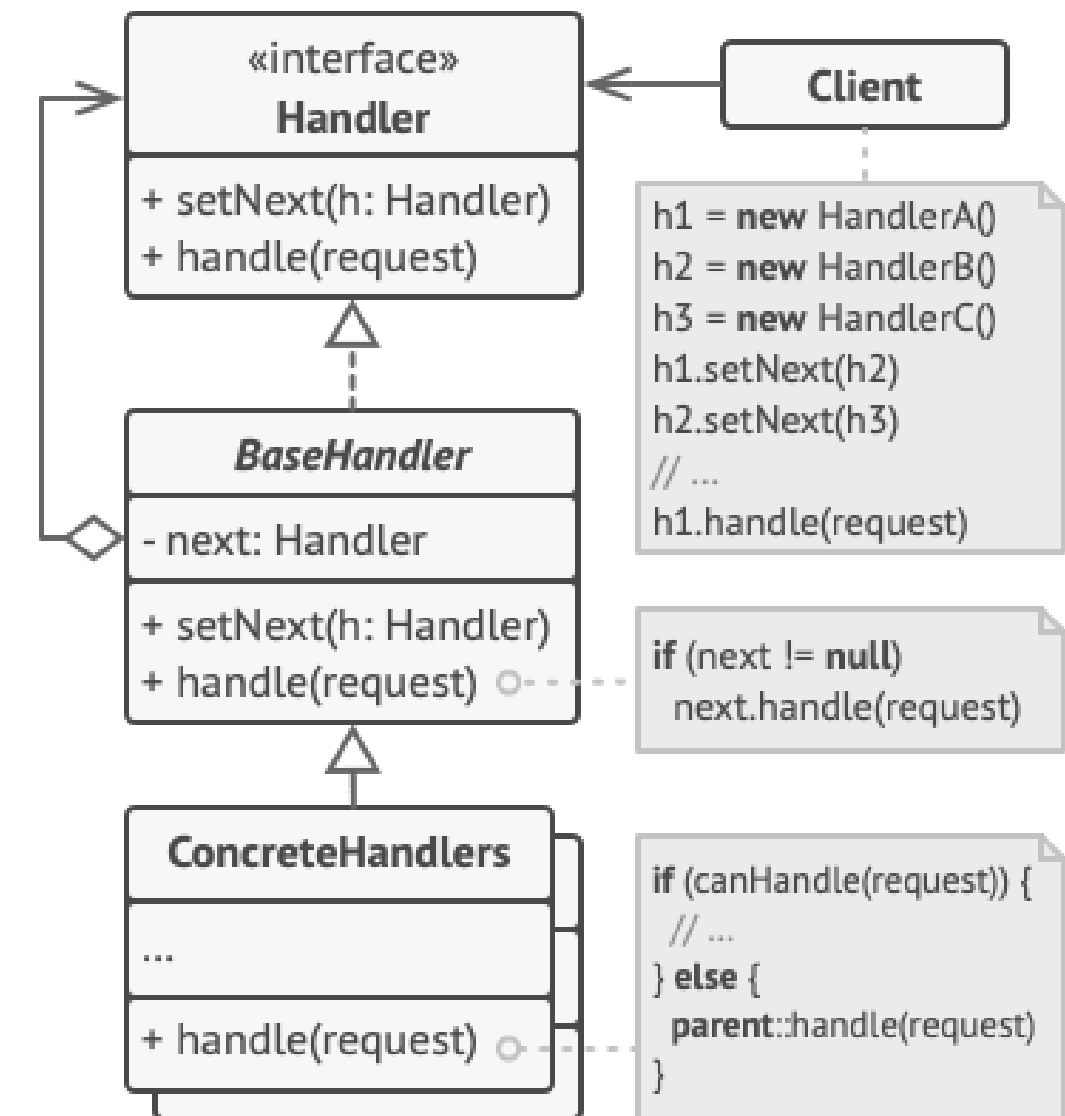
Dividimos a responsabilidade entre diversos objetos solitários (handlers) que farão as verificações em sequência.



# Como funciona

1 - O Handler inicial possui a interface comum a todos os Handlers Concretos. Ele recebe a requisição e repassa aos próximos Handlers.

2 - Os Handlers Concretos tentam resolver a requisição ou repassam para os próximos Handlers, sucessivamente.



# Exemplo de código

# 1. A Interface do Manipulador

```
class Aprovador(ABC):
    def __init__(self):
        self.proximo = None

    def definir_proximo(self, proximo):
        self.proximo = proximo
        return proximo

    @abstractmethod
    def processar(self, valor):
        if self.proximo:
            return self.proximo.processar(valor)
        print("Fim da linha: Ninguém aprovou essa compra.")
```

# 2. Manipuladores Concretos

```
class Gerente(Aprovador):
    def processar(self, valor):
        if valor <= 1000:
            print(f"Gerente aprovou a compra de R$ {valor}")
        else:
            super().processar(valor)
```

```
class Diretor(Aprovador):
    def processar(self, valor):
        if valor <= 5000:
            print(f"Diretor aprovou a compra de R$ {valor}")
        else:
            super().processar(valor)

class CEO(Aprovador):
    def processar(self, valor):
        print(f"CEO aprovou a compra de R$ {valor} (Decisão final)")
```

# --- USO DO CLIENTE ---

# Montando a corrente

```
gerente = Gerente()
diretor = Diretor()
ceo = CEO()
```

```
gerente.definir_proximo(diretor).definir_proximo(ceo)
```

# Enviando solicitações para o início da corrente

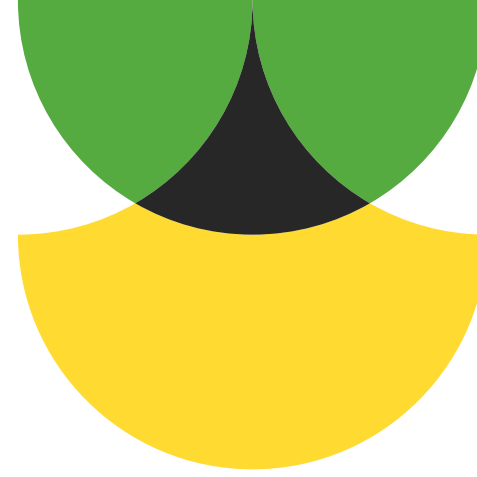
```
gerente.processar(500)    # Resolvido pelo Gerente
gerente.processar(2500)  # Passa pelo Gerente, aprovado pelo Diretor
gerente.processar(10000) # Passa por todos, aprovado pelo CEO
```

# Vantagens:

- Permite controle da ordem do tratamento de pedidos.
- Obedece o princípio de responsabilidade única.
- Obedece o princípio aberto/ fechado.

# Desvantagens:

- Alguns pedidos podem acabar sem tratamento.



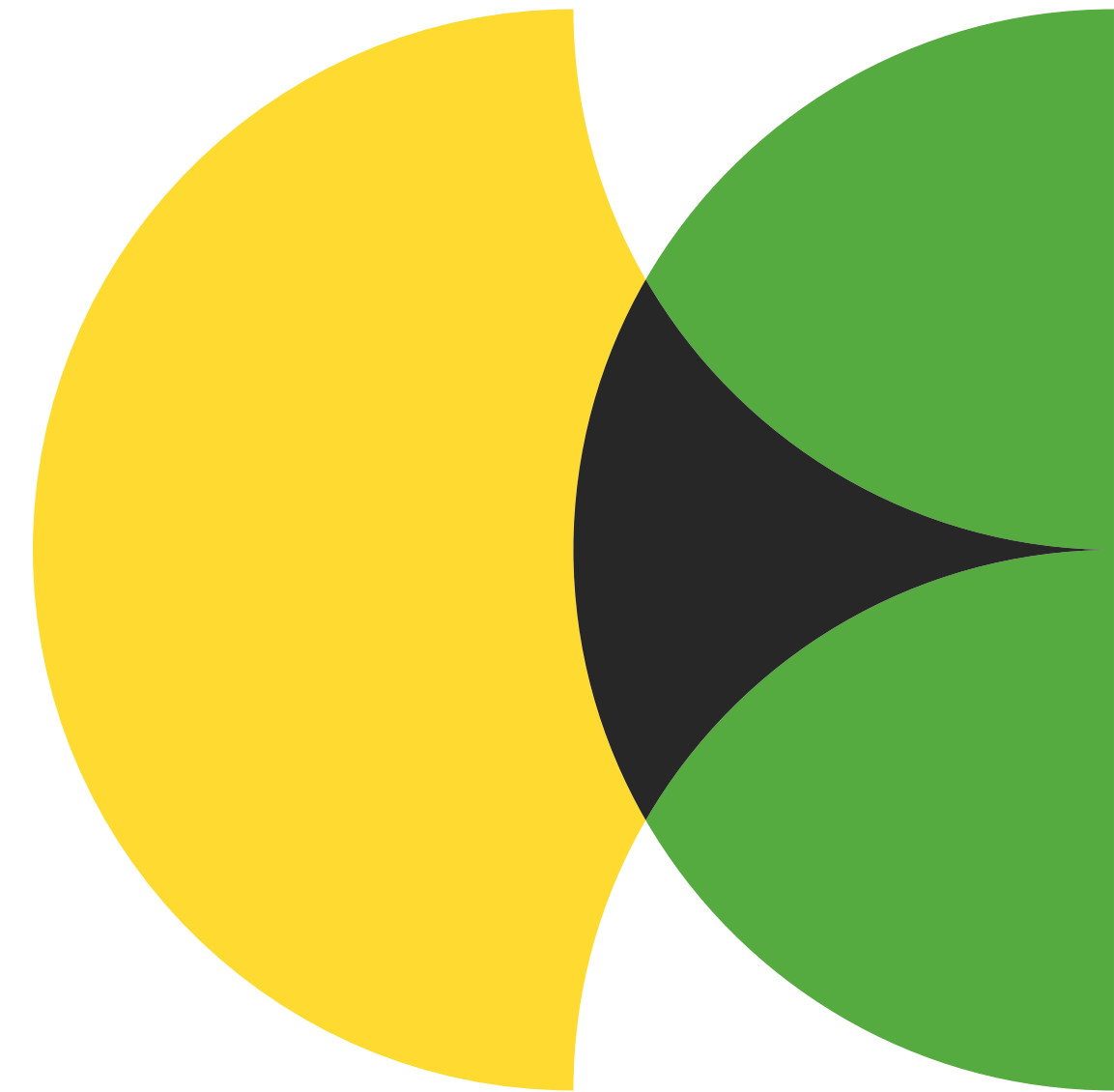




# Command

Também conhecido como: Comando, Ação,  
Action, Transação, Transaction

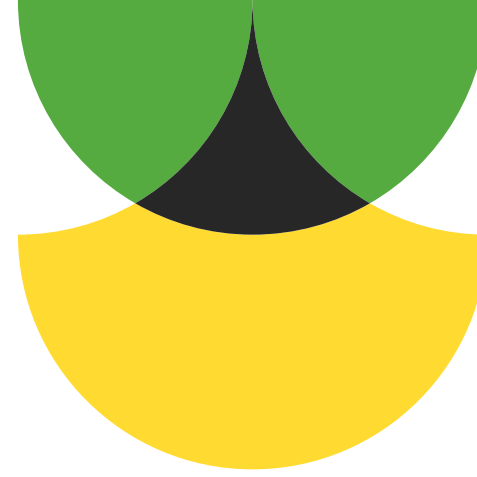
É o padrão de projeto que transforma um pedido em um objeto independente contendo toda a informação sobre o pedido.





# O problema:

Uma classe que será usada para executar um comando pode resultar em excesso de subclasses e redundância, dificultando a legibilidade e manutenção do código.



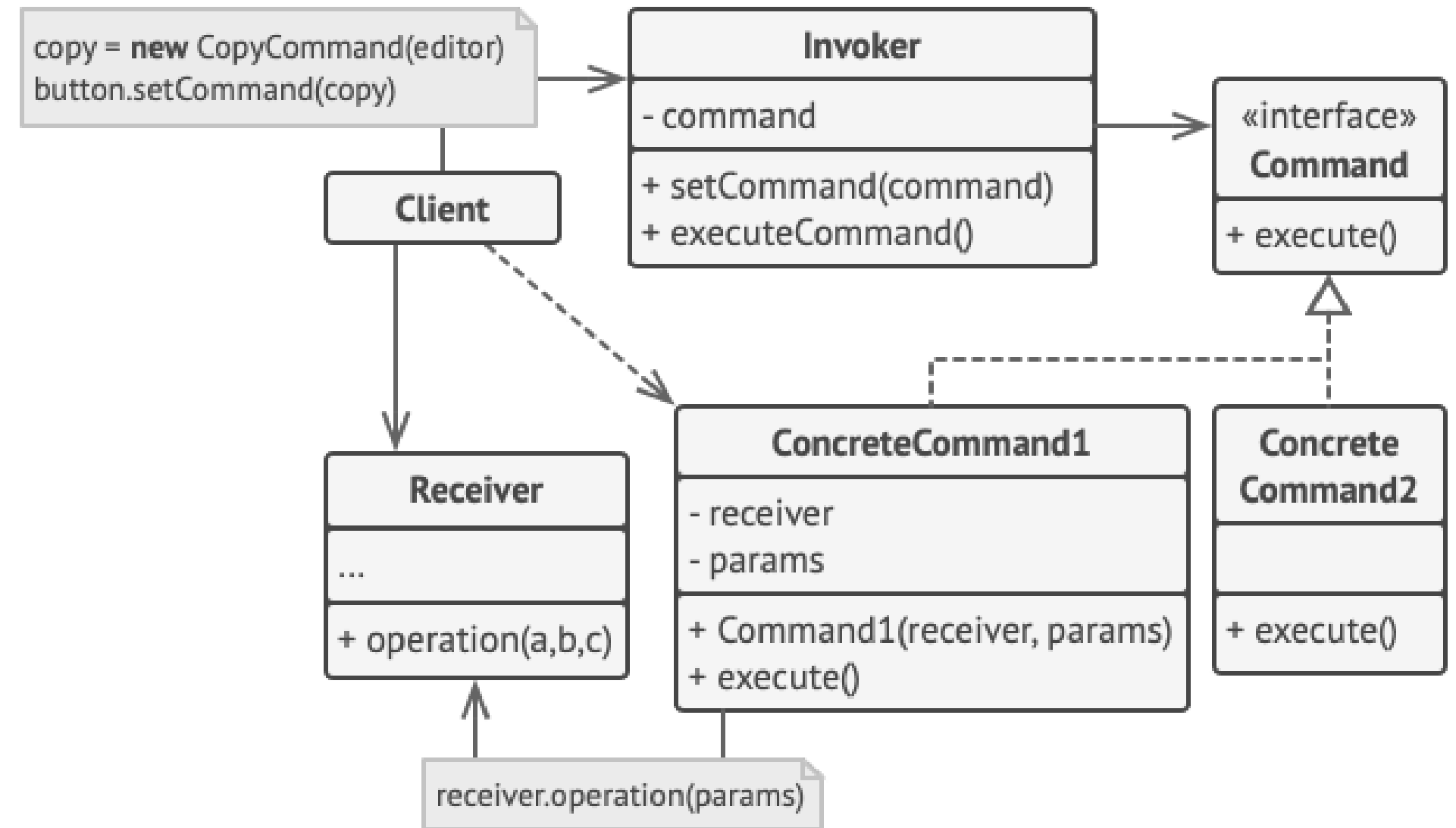
# A solução:

Fazemos com que as classes apenas tenham acesso a um objeto Command, que vai acessar a lógica de negócio sem a expor.



# Como funciona

- 1 - Criamos uma interface Comando que vai ter o método de execução.
- 2 - Criamos classes concretas que vão usar o Comando para acessar o remetente(opcional) ou o destinatário.
- 3 - Criamos o objeto destinatário (receiver), que vai fazer o trabalho ao recebe o comando.
- 4 - Usamos o método comando para acessar o destinatário.



# Exemplo de código

```
# 1. Interface Command
class Comando(ABC):
    @abstractmethod
    def executar(self): pass
    @abstractmethod
    def desfazer(self): pass

# 2. Receiver (Quem faz o trabalho)
class Luz:
    def ligar(self): print("Luz acesa!")
    def desligar(self): print("Luz apagada!")

# 3. Comando Concreto
class ComandoLigarLuz(Comando):
    def __init__(self, luz: Luz):
        self.luz = luz

    def executar(self):
        self.luz.ligar()

    def desfazer(self):
        self.luz.desligar()

# 4. Invoker (O Controle Remoto)
class ControleRemoto:
    def __init__(self):
        self.historico = []

    def pressionar_botao(self, comando: Comando):
        comando.executar()
        self.historico.append(comando)

    def pressionar_desfazer(self):
        if self.historico:
            comando = self.historico.pop()
            comando.desfazer()

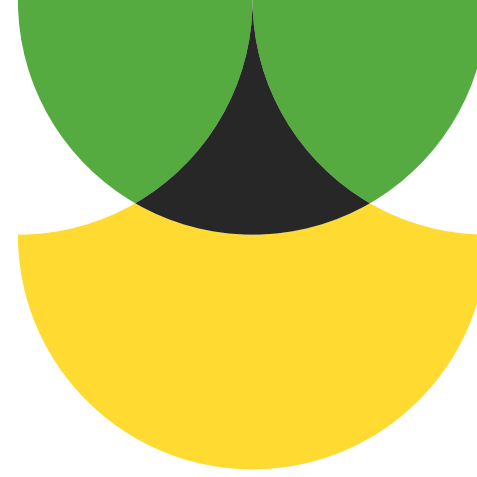
# --- USO ---
sala_luz = Luz()
controle = ControleRemoto()

# Criamos o comando e passamos para o controle
ligar = ComandoLigarLuz(sala_luz)

controle.pressionar_botao(ligar) # Saída: Luz acesa!
controle.pressionar_desfazer()  # Saída: Luz apagada!
```

# Vantagens:

- Permite implementar fazer/desfazer.
- Permite adiar a execução de operações.
- Permite juntar vários comandos simples em um complexo.
- Obedece o princípio de responsabilidade única.
- Obedece o princípio aberto/fechado.



# Desvantagens:

- O código pode ficar mais complexo.



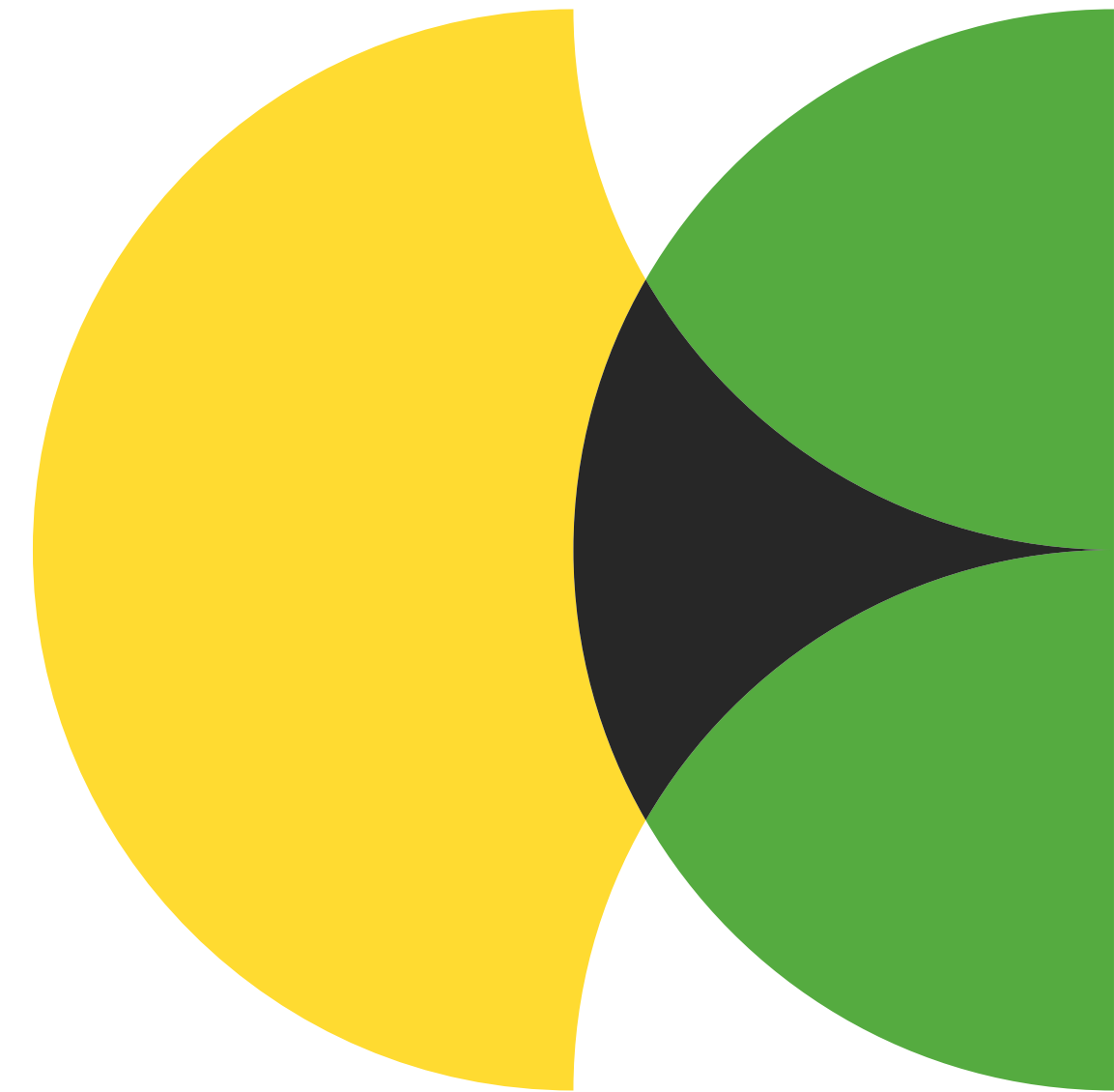


# Iterator

Também conhecido como: Iterador



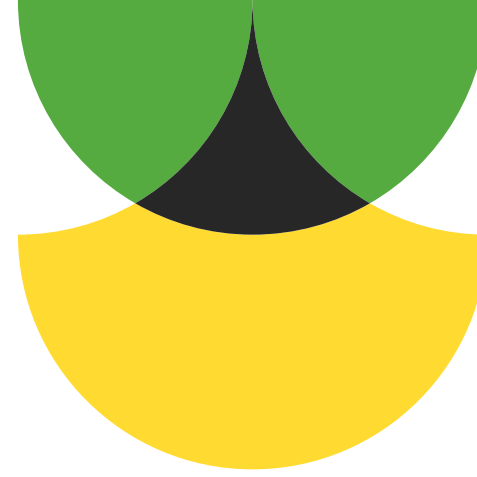
É o padrão de projeto que  
permite percorrer  
elementos de uma lista  
sem expor suas  
representações.





# O problema:

Diferentes tipos de acessos em uma estrutura de dados complexa podem ser ineficientes e causar acoplamento.



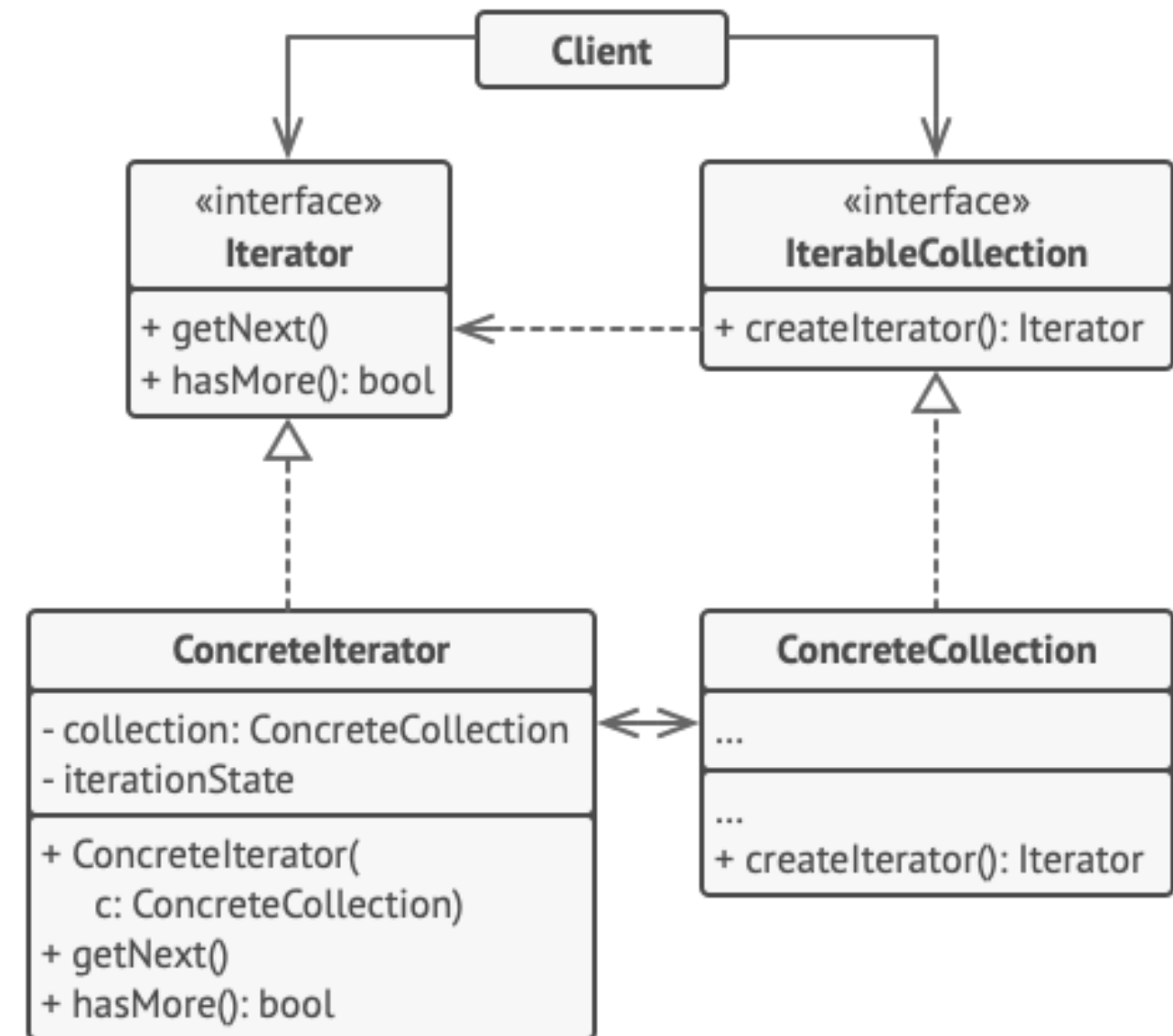
# A solução:

Tiramos o algoritmo de iteração da estrutura em si e passamos para um objeto separado chamado iterator.



# Como funciona

- 1 - Criamos uma interface Iterator.
- 2 - Criamos uma interface da coleção iterável.
- 3 - Criamos uma coleção concreta.
- 4 - Criamos o iterador concreto, que itera sobre essa coleção.



# Exemplo de código

```
class FibonacciIterator:
    def __init__(self, limite):
        self.limite = limite
        self.a = 0
        self.b = 1
        self.contador = 0

    def __iter__(self):
        # O método __iter__ deve retornar o próprio objeto iterador
        return self

    def __next__(self):
        # Verificamos se ainda estamos dentro do limite
        if self.contador < self.limite:
            resultado = self.a
            # Lógica de Fibonacci: o próximo é a soma dos dois anteriores
            self.a, self.b = self.b, self.a + self.b
            self.contador += 1
            return resultado
        else:
            # Quando o limite é atingido, levantamos esta exceção
            raise StopIteration

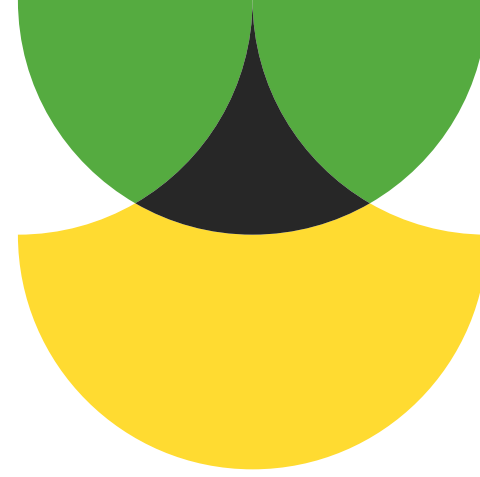
# --- Uso do Código ---

# Queremos os primeiros 10 números da sequência
fib = FibonacciIterator(10)

print("Percorrendo a sequência:")
for num in fib:
    print(num, end=" ")
```

# Vantagens:

- Permite iterar sobre cada coleção em paralelo.
- Permite atrasar uma iteração quando necessário.
- Obedece o princípio de responsabilidade única.
- Obedece o princípio aberto/fechado.



# Desvantagens:

- Pode ser desnecessário em iterações simples.
- O iterator pode ser menos eficiente em alguns casos.



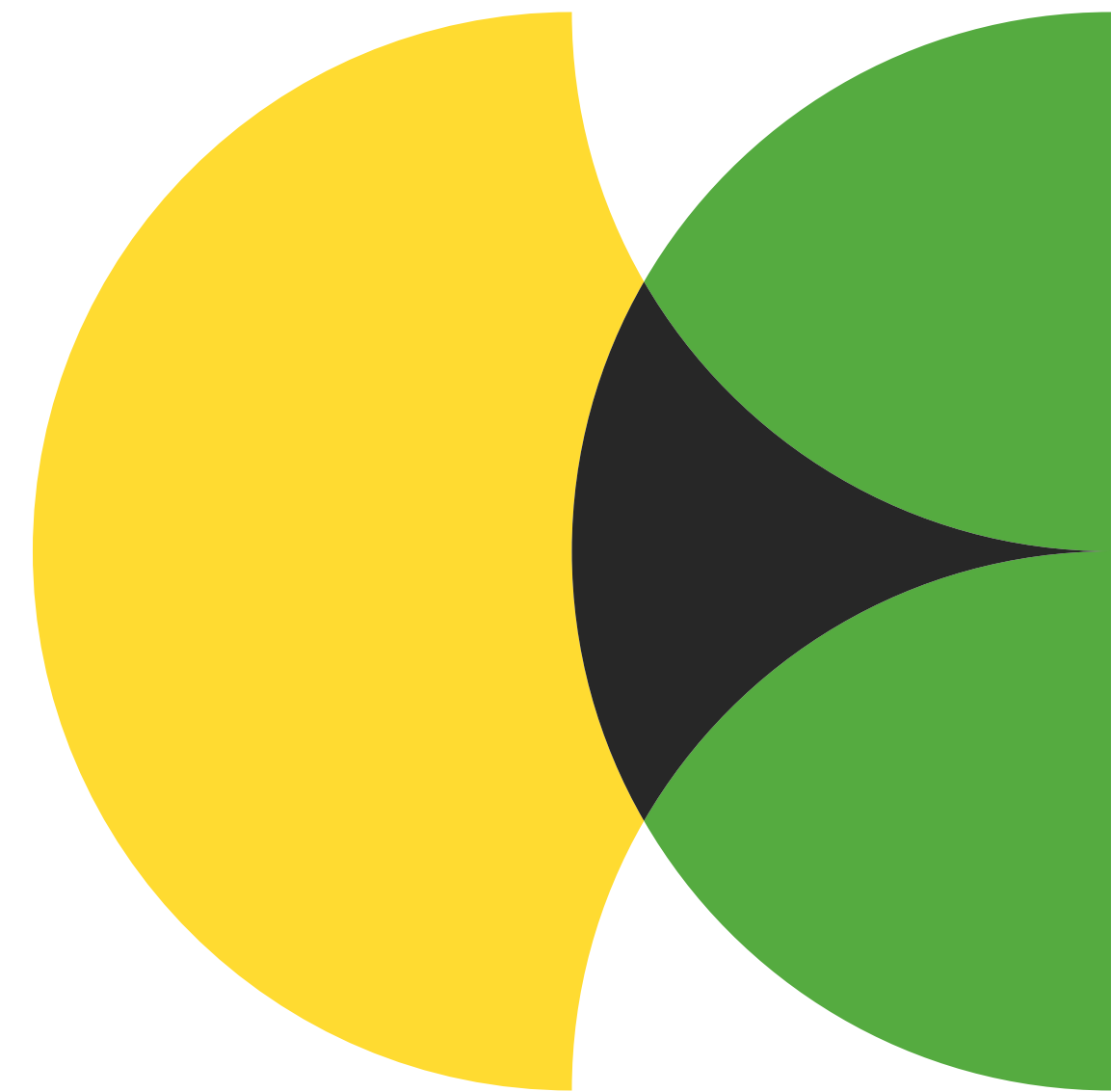


# Mediator

Também conhecido como: Mediador,  
Intermediário, Intermediary, Controlador,  
Controller



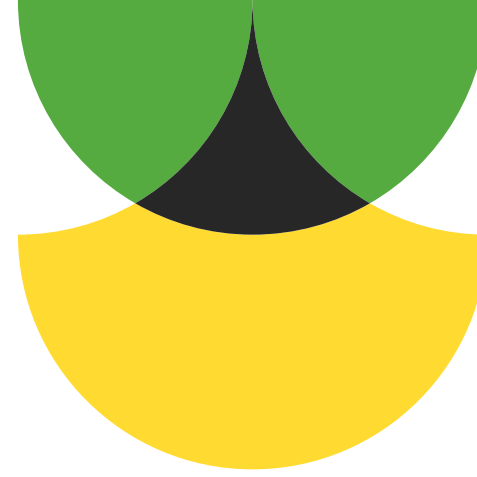
É o padrão de projeto que reduz comunicação caótica entre objetos, forçando os a se comunicar por meio de um objeto mediador.





# O problema:

Interações entre diferentes elementos de um código causam muito acoplamento, impossibilitando a reutilização e dificultando a manutenção.



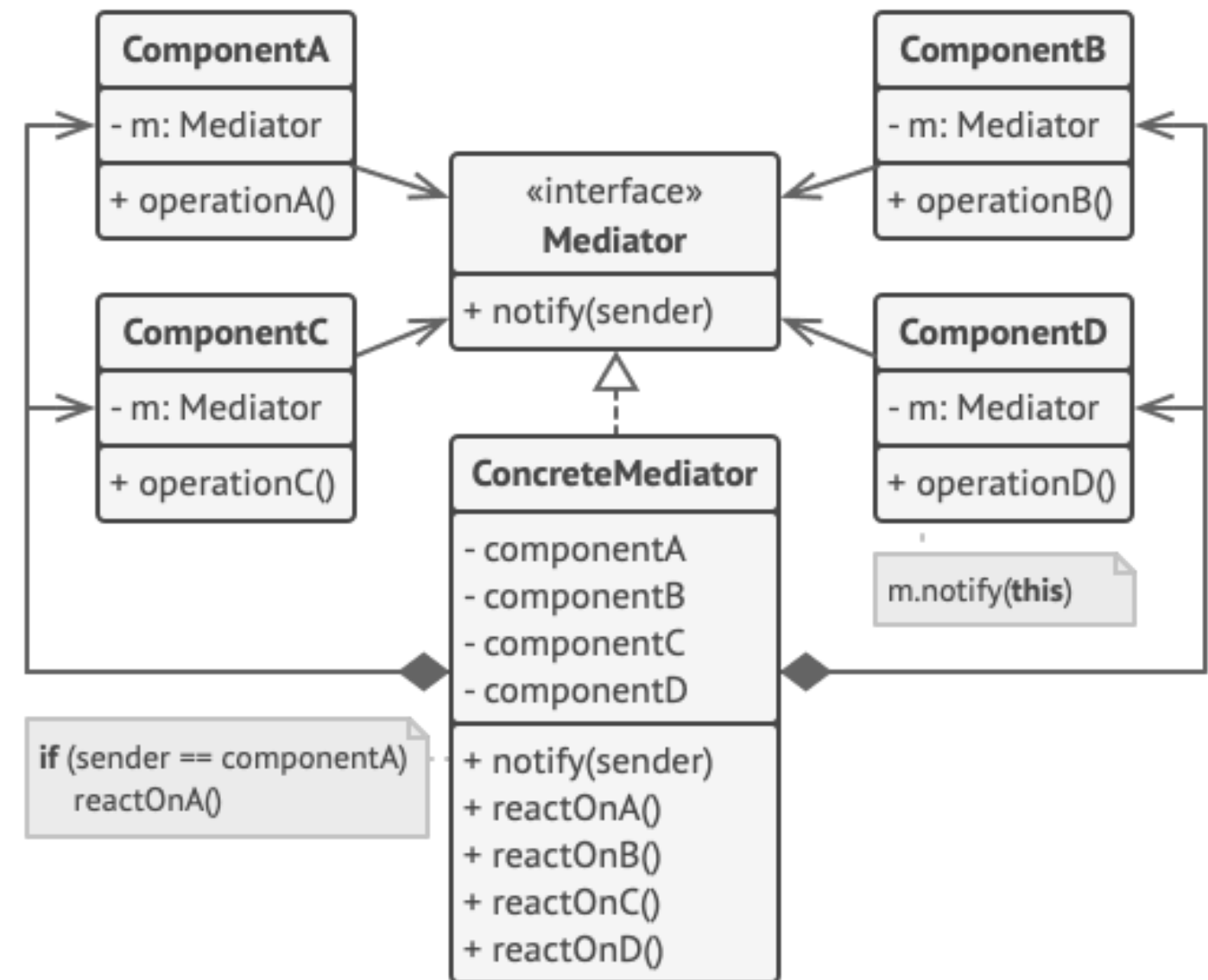
# A solução:

Criamos uma classe mediadora, que vai receber e repassar as informações entre esses elementos, reduzindo o acoplamento entre eles.



# Como funciona

- 1 - Criamos a interface mediador, que vai ter um método notificar.
- 2 - Criamos componentes que têm referência ao mediador.
- 3 - Criamos o mediador concreto, que vai receber e repassar comunicações entre os componentes.



# Exemplo de código

```
class ChatMediator:
    """O Mediador Central"""
    def __init__(self):
        self.usuarios = []

    def adicionar_usuario(self, usuario):
        self.usuarios.append(usuario)

    def enviar_mensagem(self, mensagem, remetente):
        for usuario in self.usuarios:
            # O mediador garante que o remetente não receba a própria mensagem
            if usuario != remetente:
                usuario.receber(mensagem)

class Usuario:
    """O 'Colleague' que se comunica via Mediador"""
    def __init__(self, nome, mediador):
        self.nome = nome
        self.mediador = mediador

    def enviar(self, mensagem):
        print(f"{self.nome} enviando: {mensagem}")
        self.mediador.enviar_mensagem(mensagem, self)
```

```
    def receber(self, mensagem):
        print(f"{self.nome} recebeu: {mensagem}")
```

```
# --- Execução ---
```

```
sala_de_chat = ChatMediator()
```

```
joao = Usuario("João", sala_de_chat)
```

```
maria = Usuario("Maria", sala_de_chat)
```

```
jose = Usuario("José", sala_de_chat)
```

```
sala_de_chat.adicionar_usuario(joao)
```

```
sala_de_chat.adicionar_usuario(maria)
```

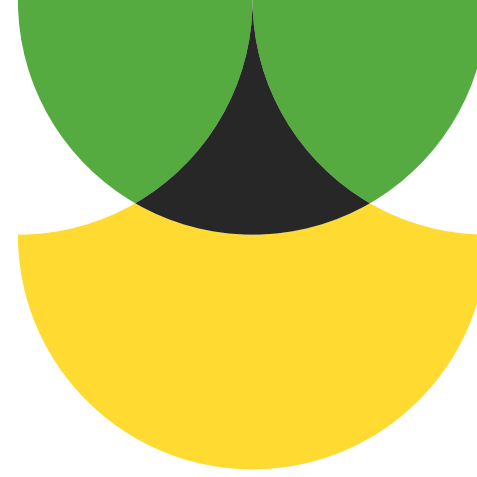
```
sala_de_chat.adicionar_usuario(jose)
```

```
joao.enviar("Olá pessoal!")
```

```
# João não sabe quem está na sala, a Torre (Mediator) resolve isso.
```

# Vantagens:

- Permite reduzir o acoplamento.
- Permite reutilizar componentes mais facilmente.
- Obedece o princípio de responsabilidade única.
- Obedece o princípio aberto/fechado.



# Desvantagens:

- O mediador pode se tornar um God Object.





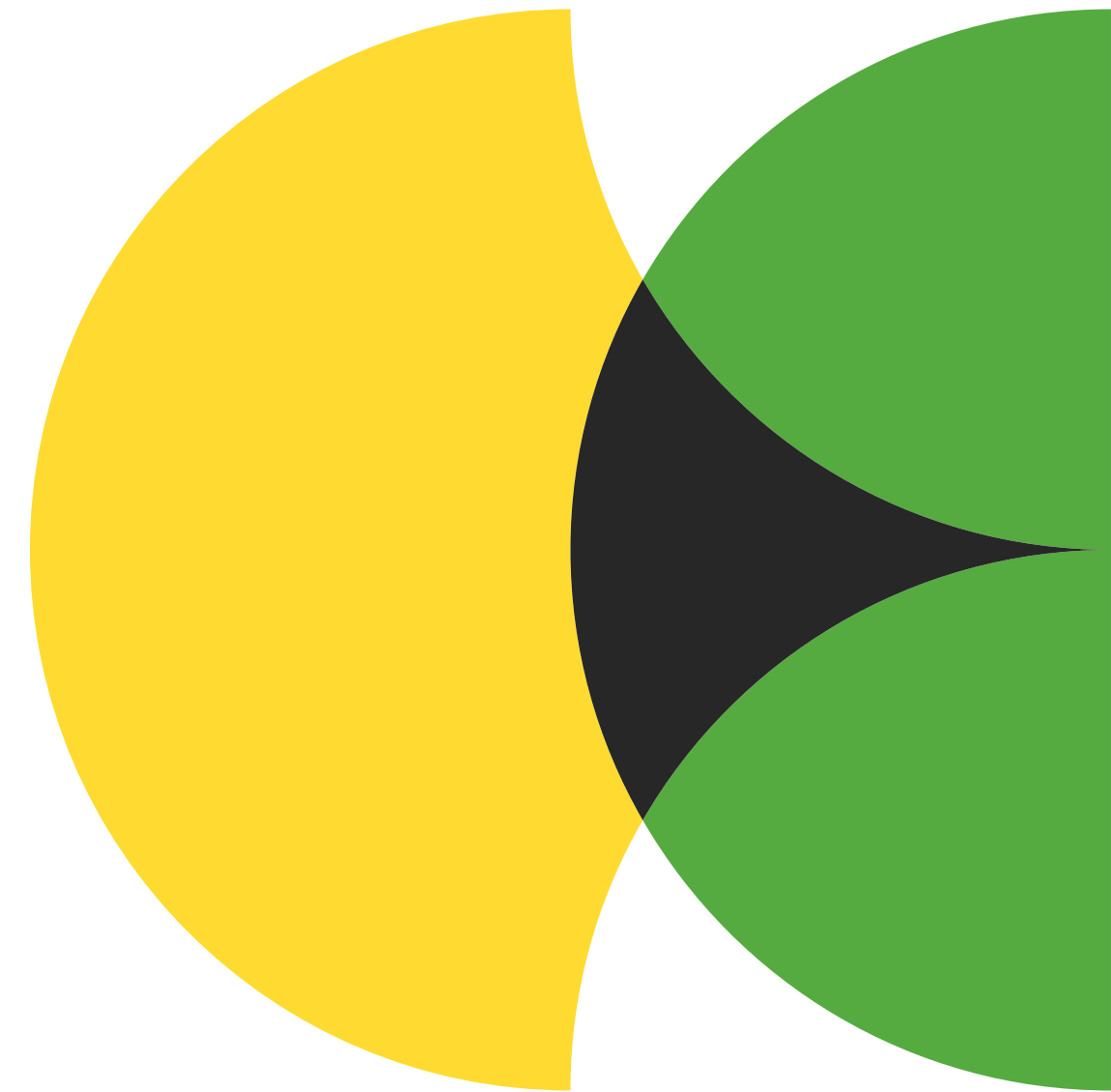


# Memento

Também conhecido como: Lembrança, Retrato, Snapshot

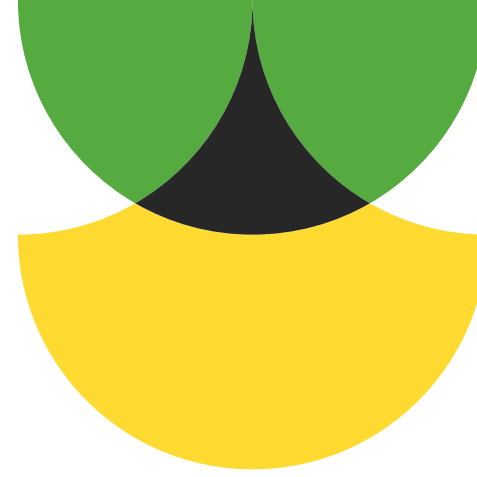


É o padrão de projeto que permite salvar e restaurar o estado anterior de um objeto sem violar o encapsulamento.



# O problema:

Fazer uma cópia do estado de todo um sistema é custoso e vai expor os atributos de um objeto a outros que não devem acessá-lo.



# A solução:

Criamos um objeto memento, que salva o estado de um objeto, permitindo acesso total apenas ao objeto original, e permitindo acesso controlado por outros objetos.

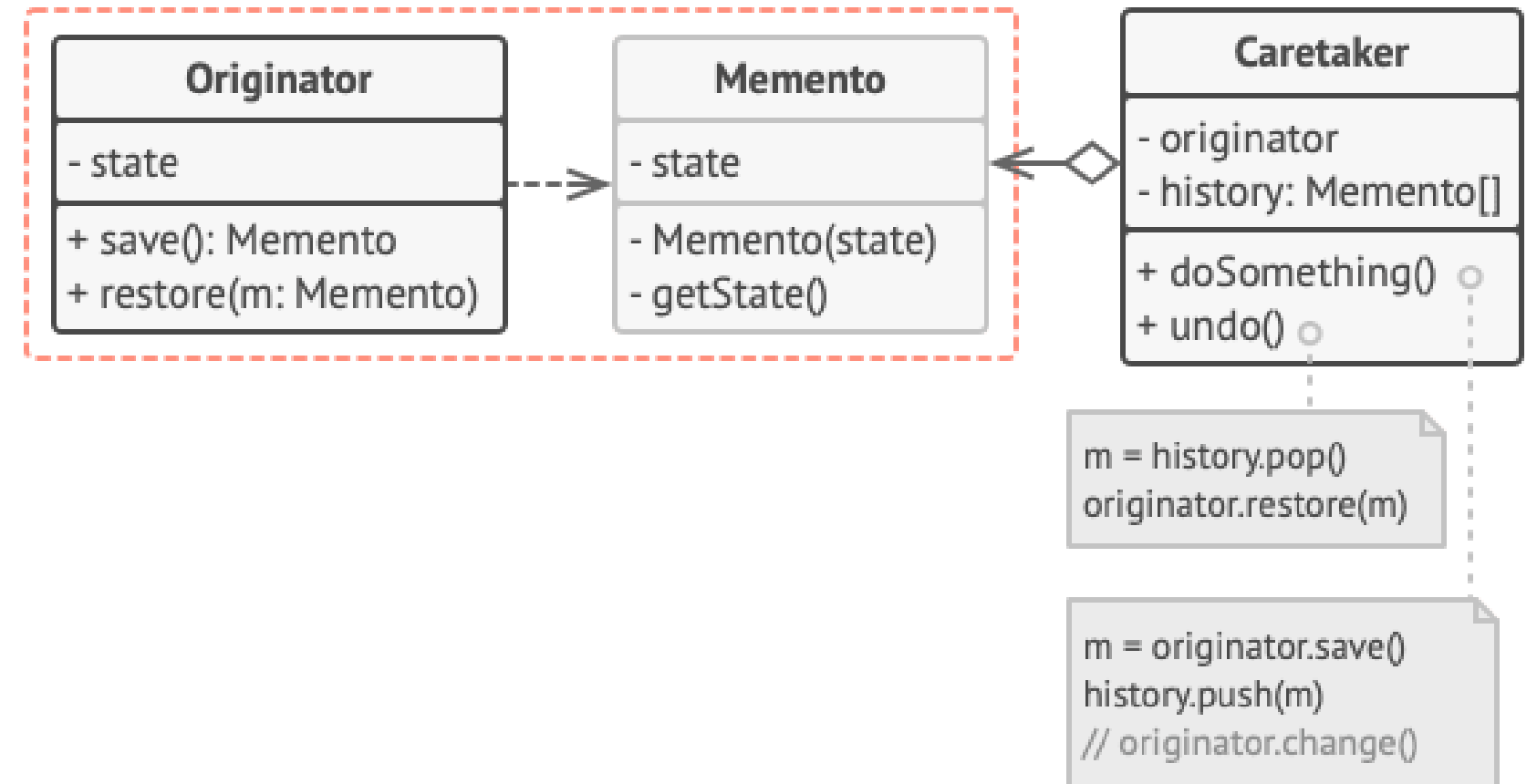


# Como funciona

1 - A classe originadora pode criar retratos do seu estado.

2 - A classe memento guarda esses estado.

3 - A classe cuidadora guarda os mementos em uma pilha para permitir recuperar estados anteriores.



# Exemplo de código

```
class Memento:
    """A 'caixa' que guarda o estado de forma privada"""
    def __init__(self, estado):
        self._estado = estado # Atributo protegido

class Editor:
    """O Originator: o objeto que queremos salvar"""
    def __init__(self):
        self.texto = ""

    def escrever(self, novo_texto):
        self.texto = novo_texto

    def salvar(self):
        print(f"-- Salvando estado: '{self.texto}' --")
        return Memento(self.texto)

    def restaurar(self, memento):
        self.texto = memento._estado
        print(f"Estado restaurado para: '{self.texto}'")

class Historico:
    """O Caretaker: ele apenas armazena a lista de mementos"""
    def __init__(self):
        self._mementos = []

    def adicionar(self, memento):
        self._mementos.append(memento)
```

```
    def desfazer(self):
        if not self._mementos:
            return None
        return self._mementos.pop()
```

```
# --- Uso Prático ---
```

```
meu_editor = Editor()
historico = Historico()
```

```
# 1. Escrevendo e salvando
```

```
meu_editor.escrever("Olá Mundo!")
historico.adicionar(meu_editor.salvar())
```

```
# 2. Fazendo besteira
```

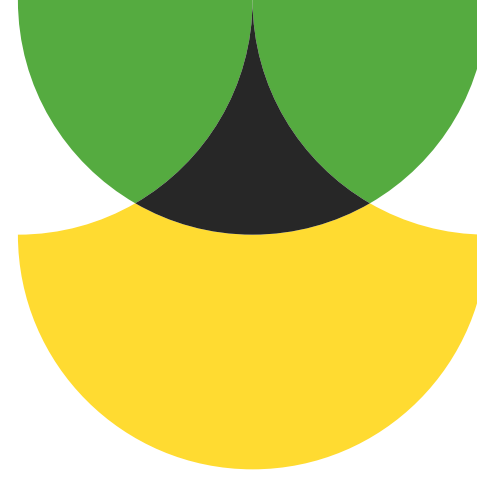
```
meu_editor.escrever("Texto errado que apaga tudo...")
```

```
# 3. Ops! Vamos desfazer
```

```
checkpoint = historico.desfazer()
meu_editor.restaurar(checkpoint)
```

# Vantagens:

- Permite salvar estados sem quebrar o encapsulamento.
- Pode simplificar o código da classe construtora deixando a classe cuidadora cuidar do histórico.



# Desvantagens:

- Pode consumir muita RAM.
- Cuidadoras tem que acompanhar o ciclo de vida da classe criadora.
- A maioria das linguagens dinâmicas não conseguem garantir que o memento se mantenha intacto.



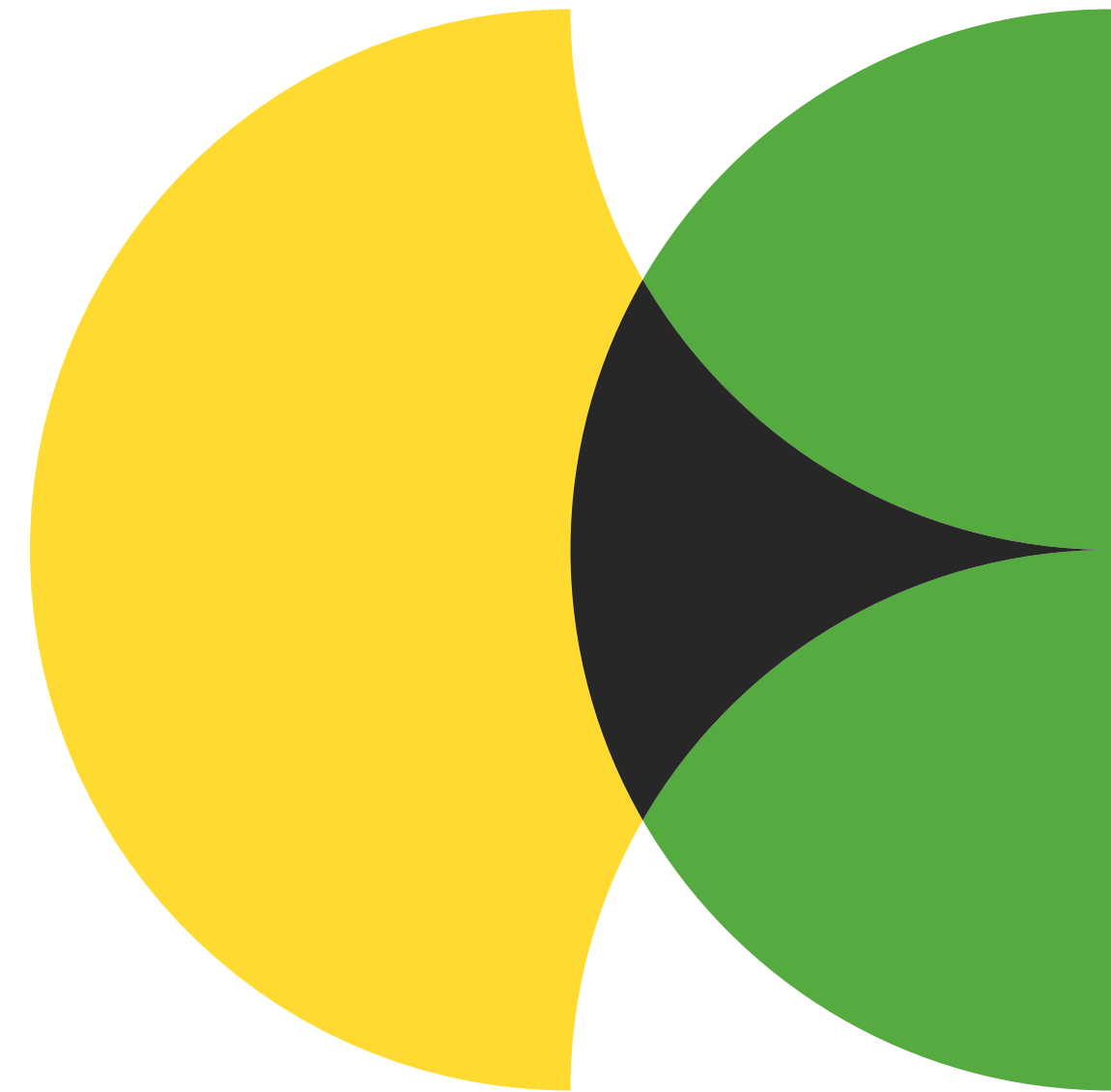




# Observer

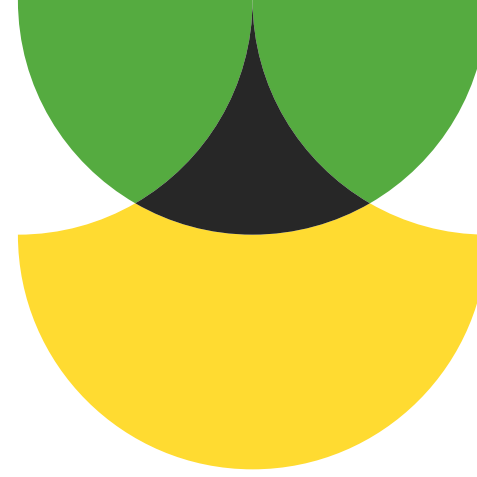
Também conhecido como: Observador, Assinante do evento, Event-Subscriber, Escutador, Listener

É o padrão de projeto que permite definir um mecanismo de assinatura para notificar múltiplos objetos sobre eventos do objeto que eles observam.



# O problema:

Verificação de estado constante, seja o objeto observado enviando notificações ou o observador requisitando atualizações, pode sobrecarregar o sistema.



# A solução:

Criamos um classe publicadora que avisa aos objetos destino sempre que algum evento ocorrer.



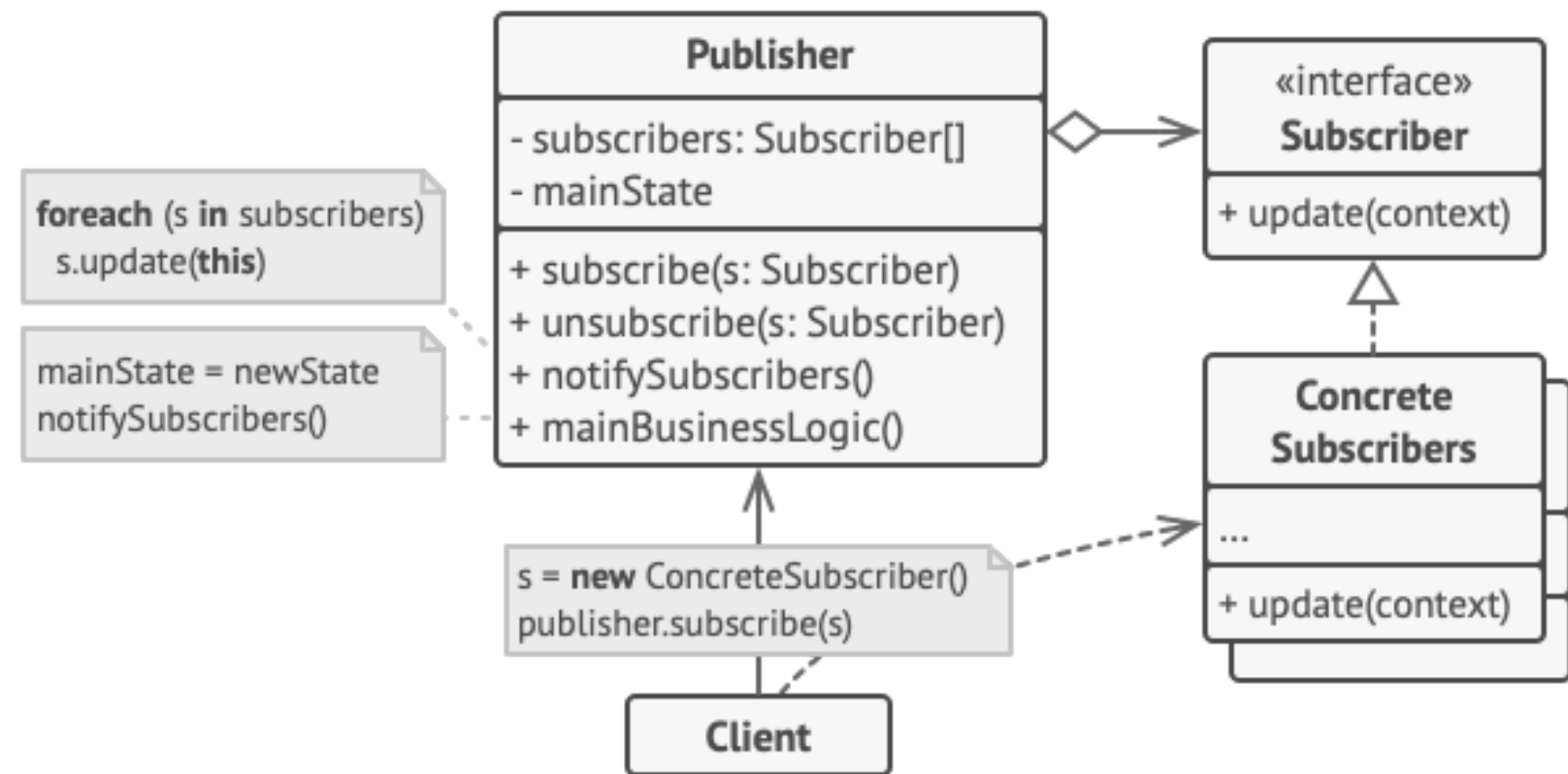


# Como funciona

1 - Criamos uma classe publicadora, que vai ter um atributo para guardar assinantes.

2 - Adicionamos os assinantes.

3 - A classe publicadora vai avisar aos assinantes sempre que o evento escolhido ocorrer.



# Exemplo de código

```
class CanalYouTube:
    """O Sujeito (Subject)"""
    def __init__(self, nome):
        self.nome = nome
        self._inscritos = []

    def inscrever(self, seguidor):
        self._inscritos.append(seguidor)

    def desinscrever(self, seguidor):
        self._inscritos.remove(seguidor)

    def postar_video(self, titulo):
        print(f"\n📺 {self.nome} postou: {titulo}")
        self._notificar_todos(titulo)

    def _notificar_todos(self, titulo):
        for inscrito in self._inscritos:
            inscrito.atualizar(self.nome, titulo)

class Seguidor:
    """O Observador (Observer)"""
    def __init__(self, nome):
        self.nome = nome

    def atualizar(self, canal, titulo):
        print(f"🔔 Olá {self.nome}, tem vídeo novo no canal {canal}: '{titulo}'")

# --- Uso Prático ---
meu_canal = CanalYouTube("DevDicas")

# Criando observadores
pedro = Seguidor("Pedro")
maria = Seguidor("Maria")

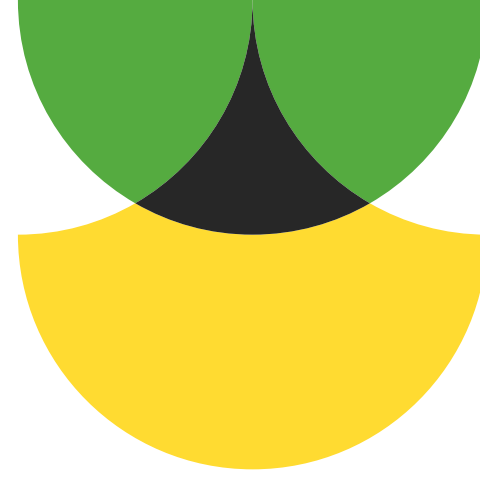
# Assinando o canal
meu_canal.inscrever(pedro)
meu_canal.inscrever(maria)

# O evento acontece!
meu_canal.postar_video("Aprenda Design Patterns")
```



# Vantagens:

- Permite estabelecer objetos durante a execução.
- Obedece o princípio aberto/fechado.



# Desvantagens:

- Assinantes são notificados em ordem aleatória.

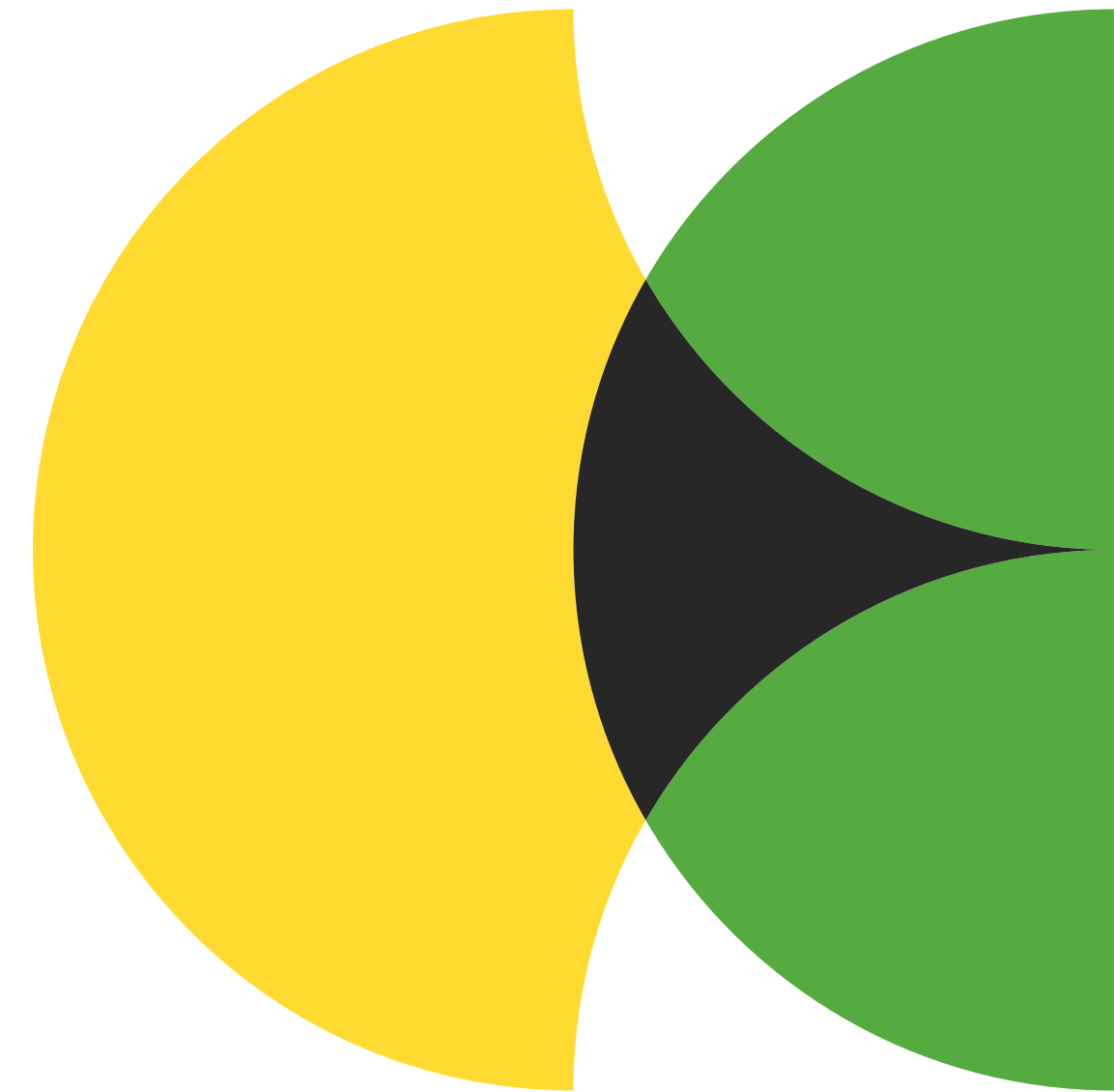




# State

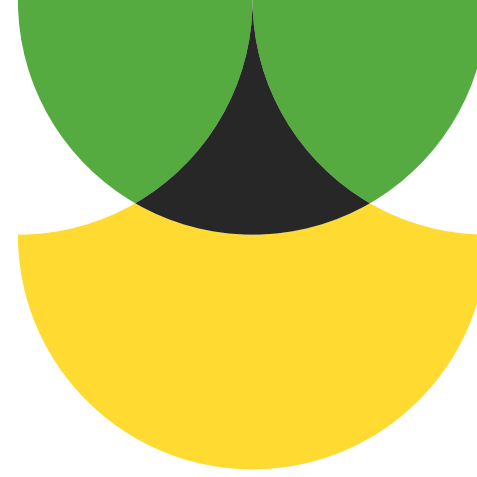
Também conhecido como: Estado

É o padrão de projeto que permite a um objeto mudar de comportamento quando seu estado interno muda.



# O problema:

Fazer um objeto mudar seu comportamento em diferentes contextos faz com que haja muitos if/else, resultando num código de difícil manutenção.



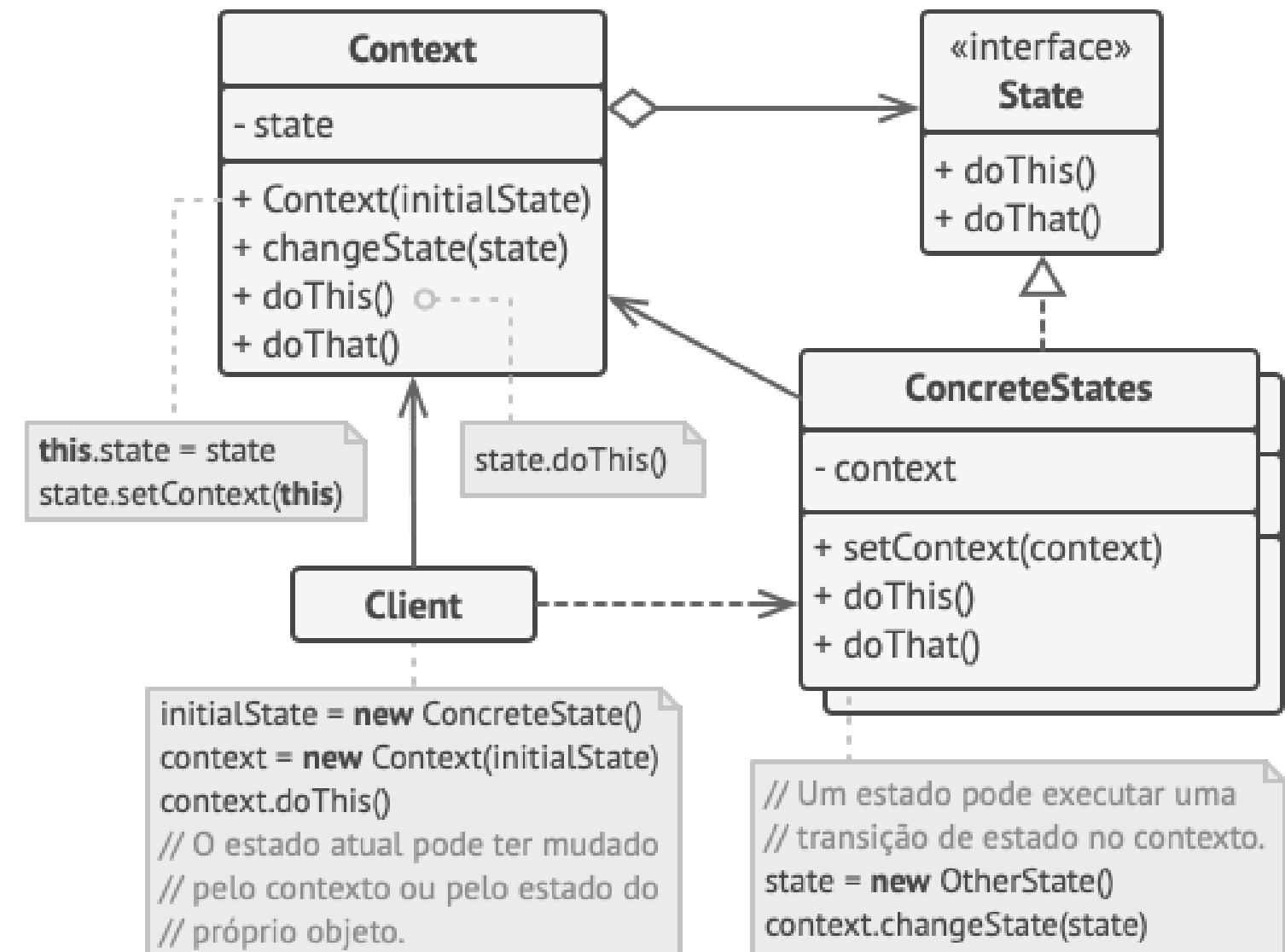
# A solução:

Mudamos todos os estados possíveis para diferentes classes e criamos um atributo que guarda esse estado. Quando um objeto muda de estado, esse atributo muda de classe.



# Como funciona

- 1 - É criado um contexto, que controla a troca de estados.
- 2 - É criada a interface estado, e dela são criados estados concretos.
- 3 - Os estados concretos e o objeto de contexto se comunicam para causar a troca de estado.





# Exemplo de código

# 1. A Interface que define o que todos os estados devem saber fazer

```
class EstadoPedido(ABC):  
    @abstractmethod  
    def proximo(self, pedido):  
        pass
```

# 2. Estados Concretos

```
class AguardandoPagamento(EstadoPedido):  
    def proximo(self, pedido):  
        print("Pagamento aprovado! Preparando o pedido...")  
        pedido.set_estado(Preparando())
```

```
class Preparando(EstadoPedido):  
    def proximo(self, pedido):  
        print("Pedido pronto! Saiu para entrega...")  
        pedido.set_estado(SaiuParaEntrega())
```

```
class SaiuParaEntrega(EstadoPedido):  
    def proximo(self, pedido):  
        print("Pedido entregue com sucesso!")  
        # Aqui poderia finalizar o ciclo
```

# 3. O Contexto (O Objeto Principal)

```
class Pedido:  
    def __init__(self):  
        self.estado_atual = AguardandoPagamento()
```

```
    def set_estado(self, novo_estado):  
        self.estado_atual = novo_estado
```

```
    def avancar(self):  
        # O Pedido não sabe o que fazer, ele pergunta para o Estado Atual  
        self.estado_atual.proximo(self)
```

# --- Uso Prático ---

```
meu_jantar = Pedido()
```

```
meu_jantar.avancar() # Pagamento -> Preparando
```

```
meu_jantar.avancar() # Preparando -> Entrega
```

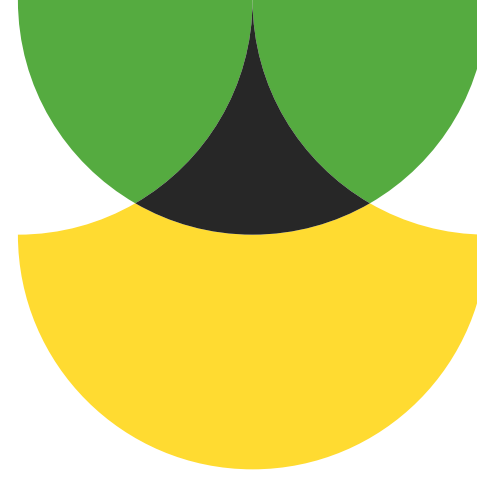
```
meu_jantar.avancar() # Entrega -> Finalizado
```

# Vantagens:

- Simplifica o código ao evitar máquinas de estado ineficientes.
- Obedece o princípio aberto/fechado.
- Obedece o princípio de responsabilidade única.

# Desvantagens:

- Pode ser desnecessário em caso de máquinas de estado simples.

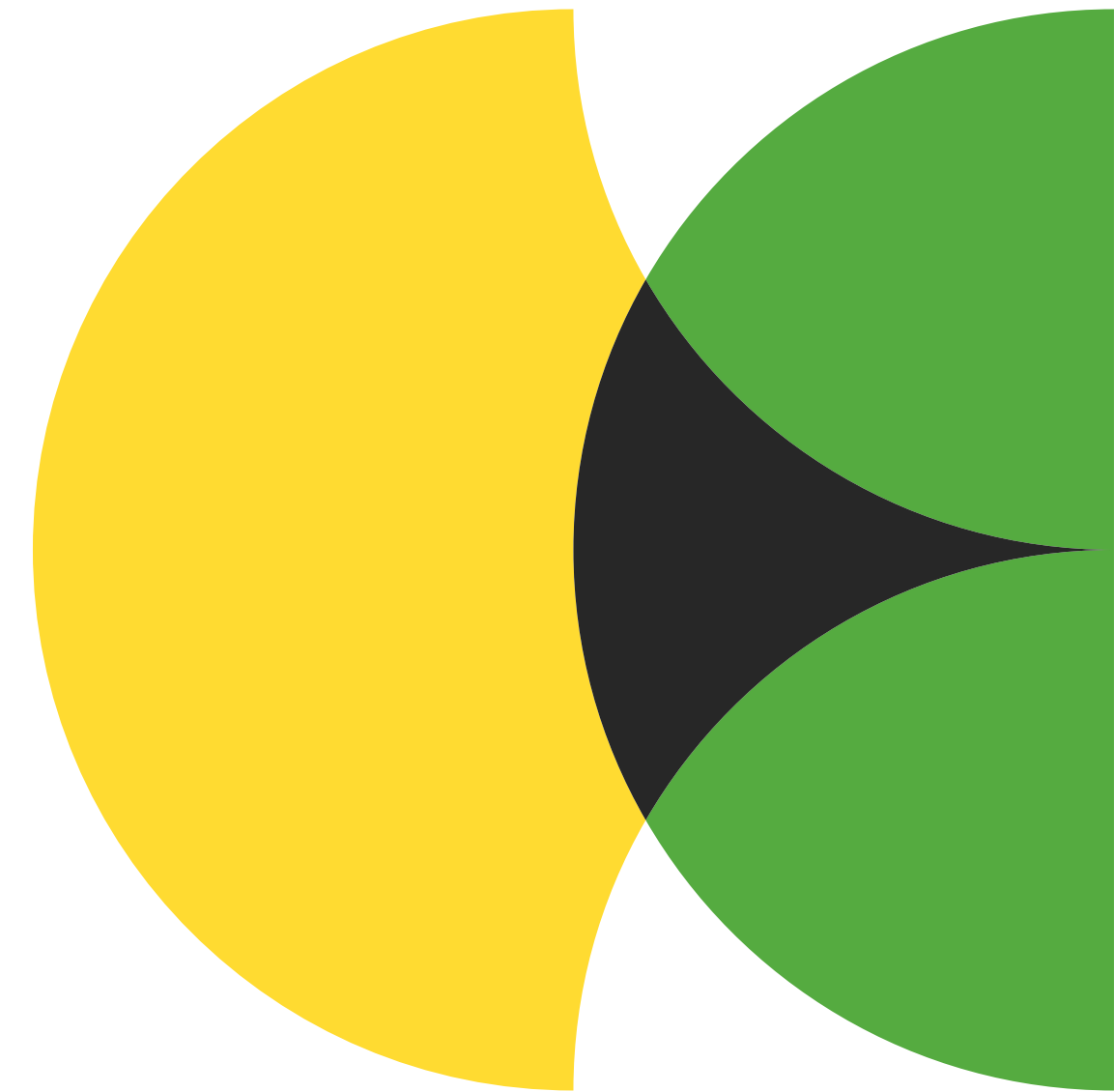




# Strategy

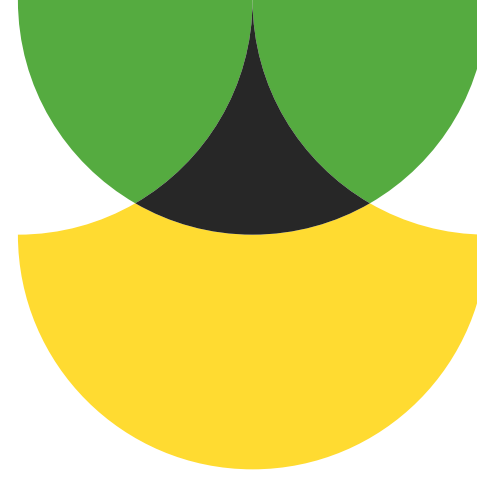
Também conhecido como: Estratégia

É o padrão de projeto que permite criar uma família de algoritmos em classes separadas e tornar seus objetos intercambiáveis.



# O problema:

Às vezes pode ser necessário que um programa faça uma coisa similar de maneiras diferentes, mas o uso de if/else tornará o código de difícil manutenção.



# A solução:

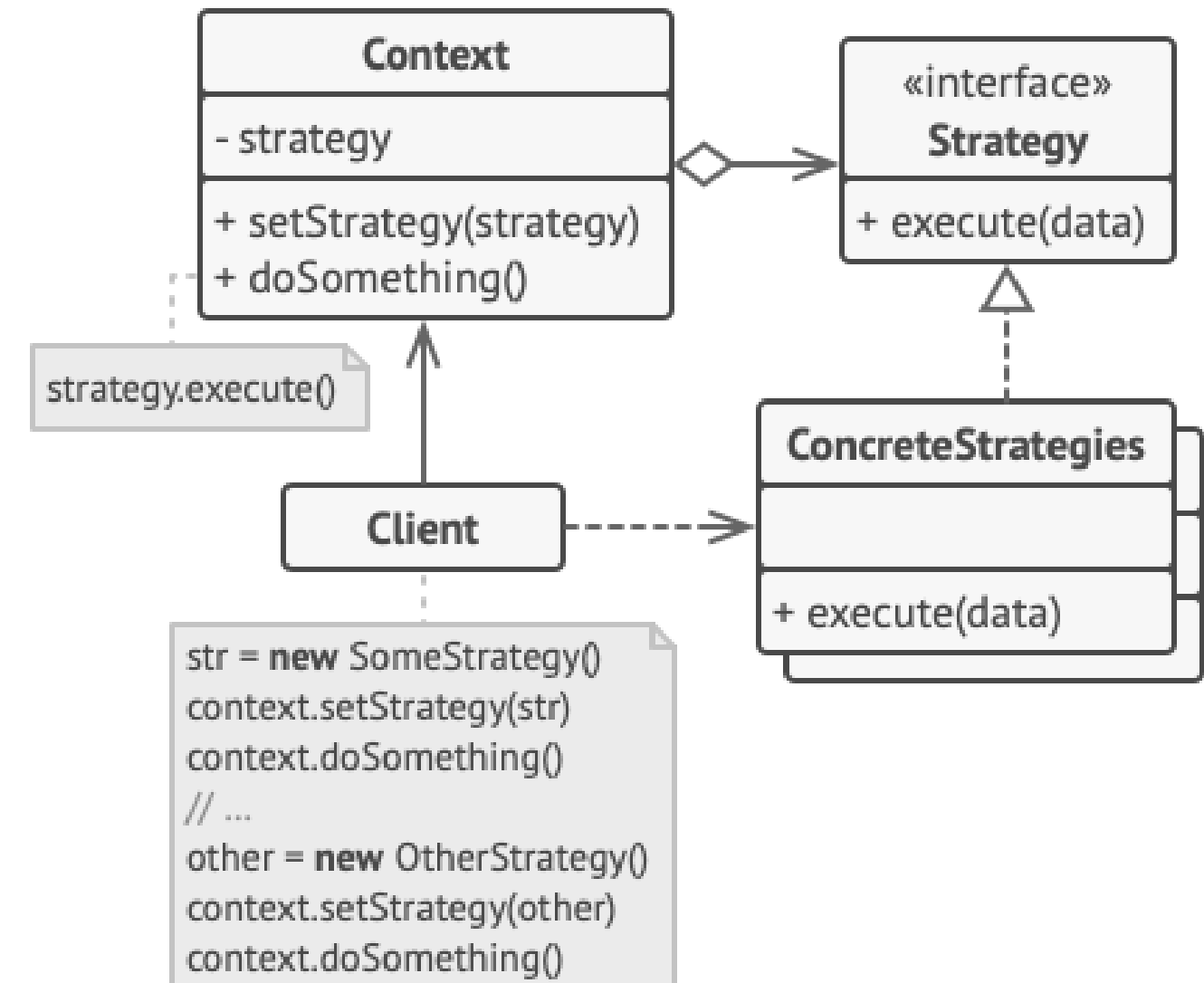
Criamos uma interface genérica que pode receber diferentes objetos (estratégias) para concluir seus objetivos.





# Como funciona

- 1 - É criado um contexto, que tem referência à interface de estratégias.
- 2 - É criada a interface estratégia, e dela são criadas estratégias concretas.
- 3 - O contexto usa a estratégia que o cliente escolher.



# Exemplo de código

```
# 1. A Interface (O contrato que todas as estratégias devem seguir)
```

```
class EstrategiaEnvio(ABC):  
    @abstractmethod  
    def calcular(self, peso):  
        pass
```

```
# 2. Estratégias Concretas (Os algoritmos específicos)
```

```
class EnvioPAC(EstrategiaEnvio):  
    def calcular(self, peso):  
        return peso * 1.5 + 10
```

```
class EnvioSedex(EstrategiaEnvio):  
    def calcular(self, peso):  
        return peso * 4.5 + 25
```

```
class EnvioGratis(EstrategiaEnvio):  
    def calcular(self, peso):  
        return 0.0
```

```
# 3. O Contexto (Quem usa a estratégia)
```

```
class CalculadoraDeFrete:  
    def __init__(self, estrategia: EstrategiaEnvio):  
        self._estrategia = estrategia  
  
    def definir_estrategia(self, nova_estrategia):  
        self._estrategia = nova_estrategia  
  
    def calcular_total(self, peso):  
        return self._estrategia.calcular(peso)
```

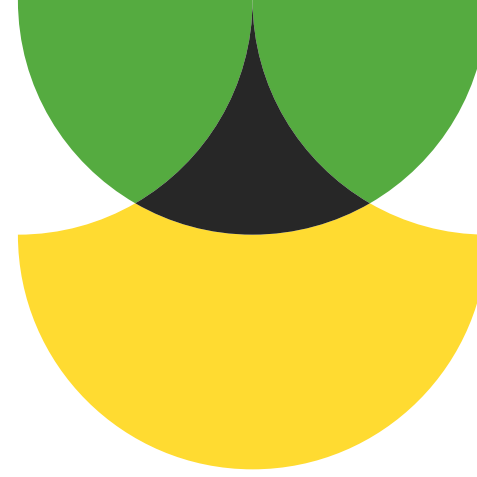
```
# --- Uso Prático ---
```

```
carrinho = CalculadoraDeFrete(EnvioPAC())  
print(f"Frete PAC: R$ {carrinho.calcular_total(2)}")
```

```
# O usuário mudou de ideia e quer Sedex  
carrinho.definir_estrategia(EnvioSedex())  
print(f"Frete Sedex: R$ {carrinho.calcular_total(2)}")
```

# Vantagens:

- Permite trocar algoritmos durante a execução.
- Permite isolar os detalhes de um algoritmo de seu código.
- Permite substituir herança por composição.
- Obedece o princípio aberto/fechado.



# Desvantagens:

- Se há poucos algoritmos, é um padrão desnecessário.
- Os clientes devem conhecer as estratégias para poder escolher.
- É desnecessário em programação funcional.

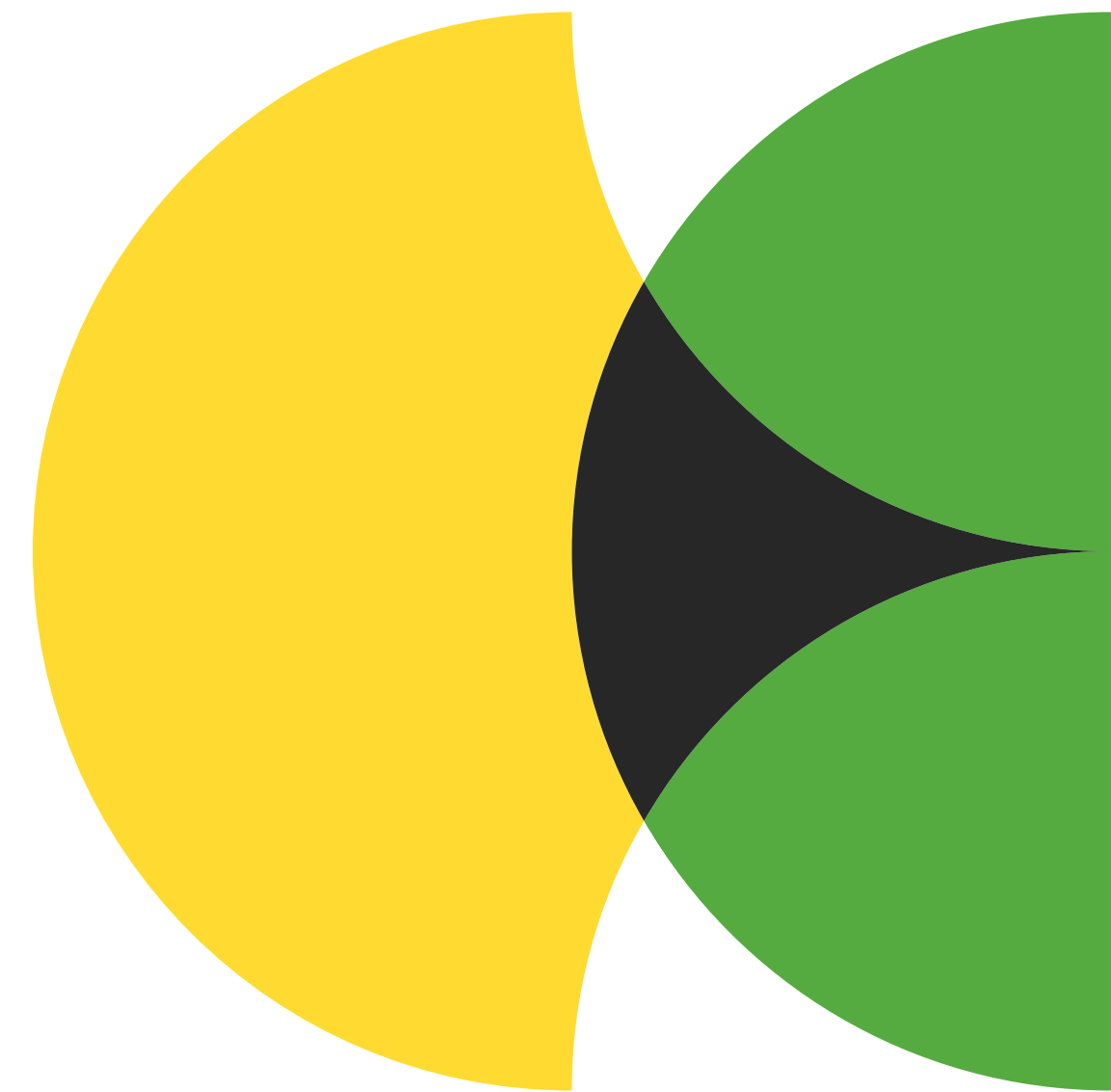




# Template Method

Também conhecido como: Método padrão

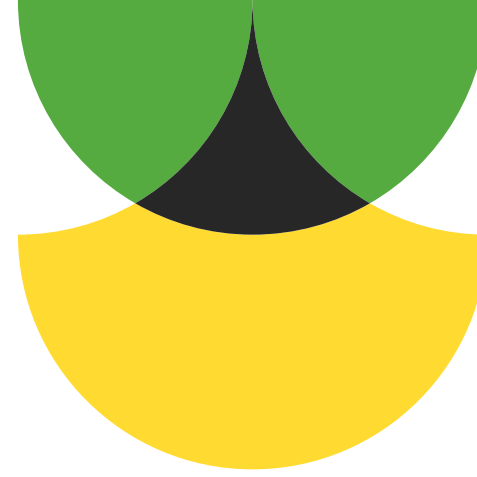
É o padrão de projeto que define o esqueleto de um algoritmo na superclasse, mas permite sobrescrever etapas sem alterar a estrutura.





# O problema:

Quando queremos fazer coisas similares que seguem etapas, criar diferentes classes vão resultar num código de baixa coesão.



# A solução:

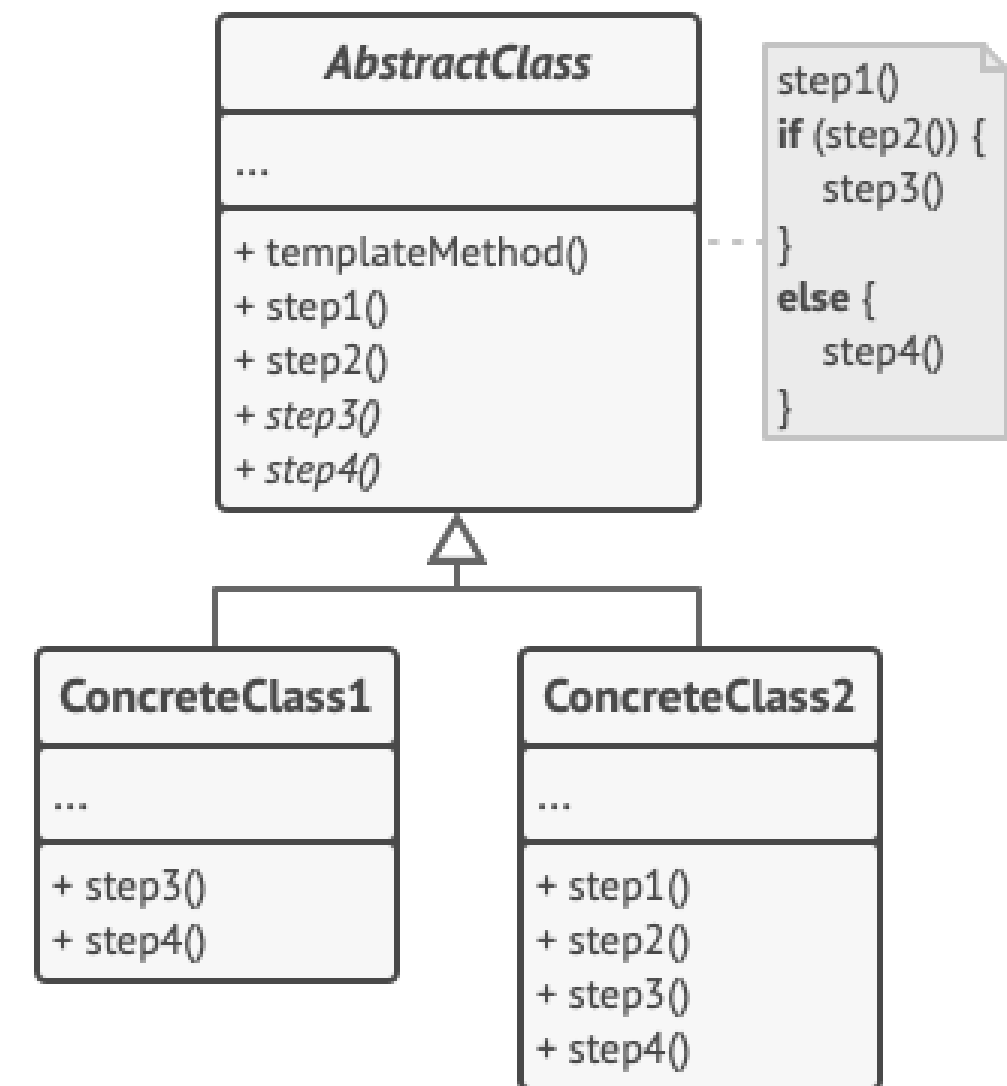
Criamos uma superclasse, com métodos abstratos ou não, e criamos subclasses que sobrescrevem apenas os métodos necessários.



# Como funciona

1 - Criamos uma classe comum, que vai ter todos os métodos a serem utilizados.

2 - As classes concretas sobrescrevem os métodos necessários.



# Exemplo de código

```
class GeradorRelatorio(ABC):
    """A Classe Mãe com o 'Esqueleto' (Template)"""

    def gerar(self):
        """O Template Method: define a ordem dos passos"""
        self.coletar_dados()
        self.formatar_cabecalho()
        self.formatar_corpo()
        self.salvar()

    def coletar_dados(self):
        # Passo fixo: todos os relatórios coletam dados do mesmo jeito
        print("📄 Coletando dados do banco de dados...")

    @abstractmethod
    def formatar_cabecalho(self):
        # Passo que cada filho DEVE implementar
        pass

    @abstractmethod
    def formatar_corpo(self):
        # Passo que cada filho DEVE implementar
        pass

    def salvar(self):
        # Passo comum com comportamento padrão
        print("💾 Salvando arquivo no servidor...")

# --- Subclasses (Filhos) ---

class RelatorioPDF(GeradorRelatorio):
    def formatar_cabecalho(self):
        print("📄 Criando cabeçalho com logo elegante em PDF")

    def formatar_corpo(self):
        print("📄 Renderizando tabelas e gráficos em PDF")

class RelatorioHTML(GeradorRelatorio):
    def formatar_cabecalho(self):
        print("🌐 Criando tag <header> em HTML")

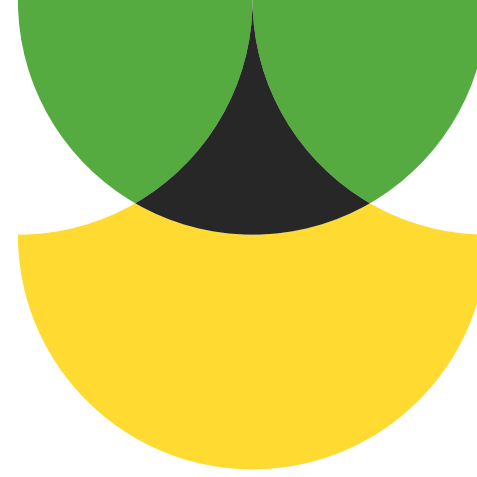
    def formatar_corpo(self):
        print("🌐 Criando estrutura <div> e <table> em HTML")

# --- Uso Prático ---
print("--- Gerando PDF ---")
pdf = RelatorioPDF()
pdf.gerar()

print("\n--- Gerando HTML ---")
html = RelatorioHTML()
html.gerar()
```

# Vantagens:

- Permite escolher quais partes do algoritmo podem ser sobrescritas.
- Permite elevar os códigos duplicados para uma superclasse.



# Desvantagens:

- Pode limitar alguns clientes.
- Pode violar o princípio de substituição de Liskov.
- É um padrão que se torna de difícil manutenção quando há muitas etapas.





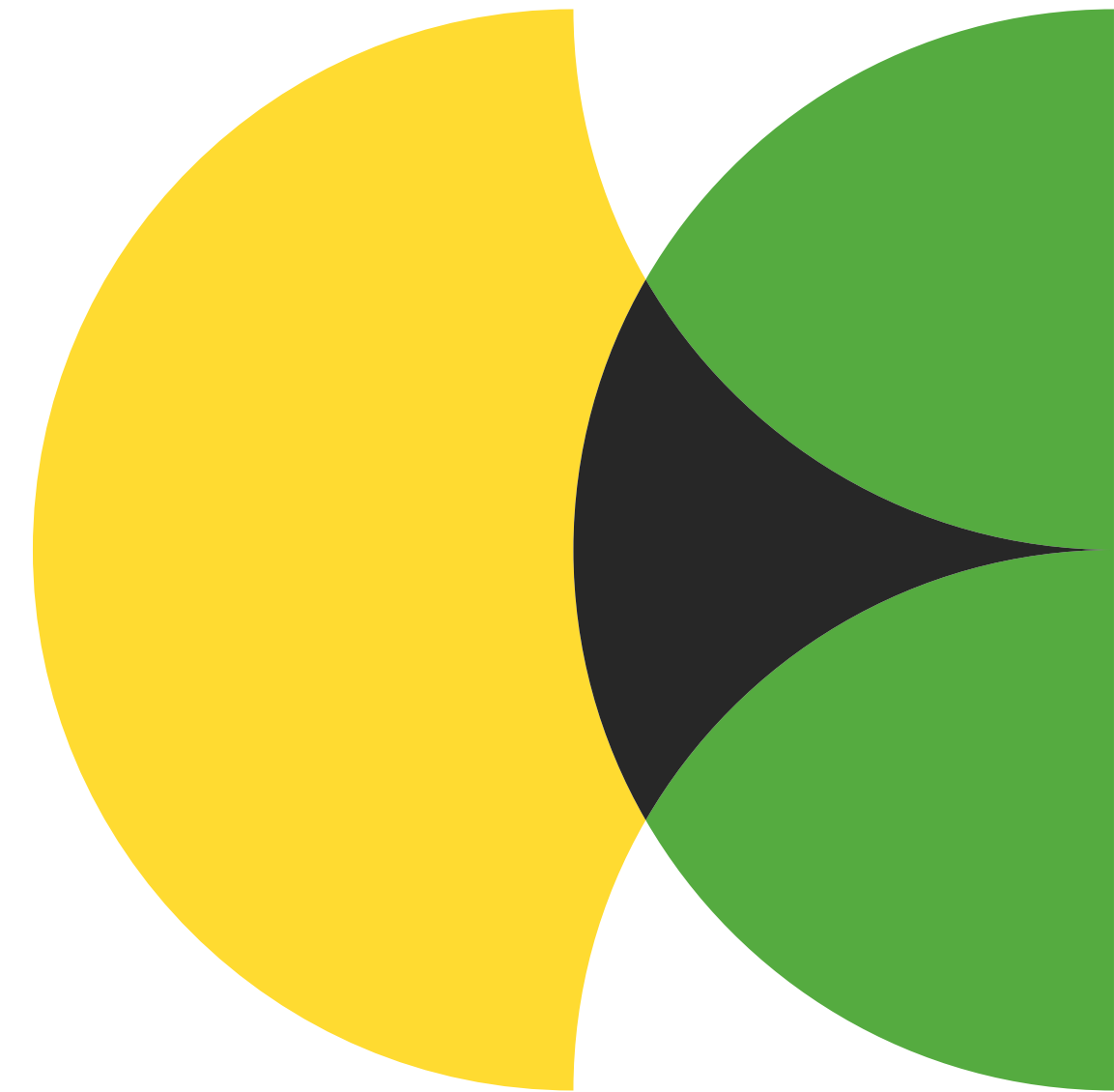


# Visitor

Também conhecido como: Visitante

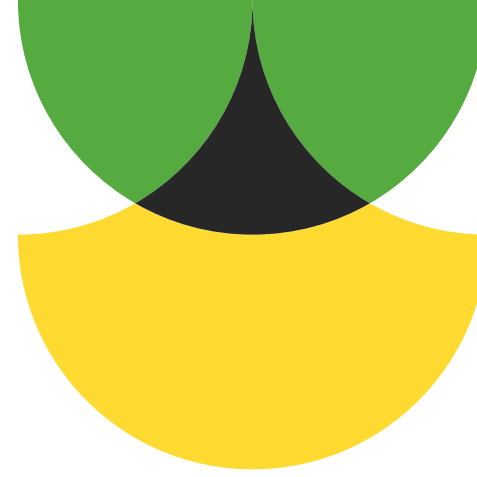


É o padrão de projeto que  
permite separar  
algoritmos dos objetos  
em que eles operam.



# O problema:

Adicionar métodos em classes estáveis pode quebrar seu código ou seu propósito.



# A solução:

Criamos um objeto visitante, e adicionamos, na classe original, apenas o método para receber esse visitante.

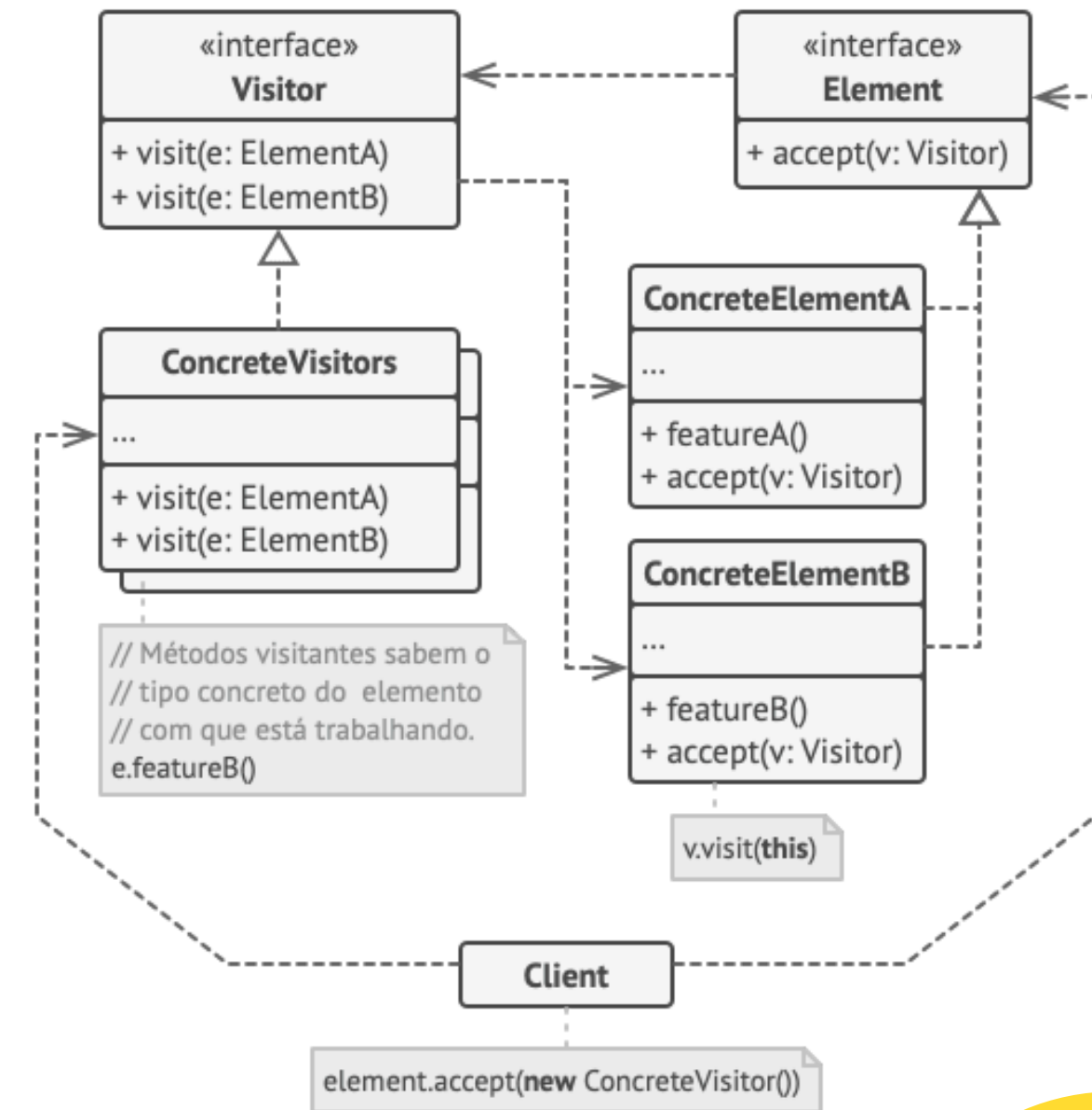


# Como funciona

1 - Criamos uma interface visitante e os visitantes concretos.

2 - Criamos a interface elemento e os elementos concretos.

3 - Os elementos concretos tem métodos para receber os visitantes.



# Exemplo de código

```
# 1. A Interface do Visitante (O que o perito sabe analisar)
class Visitante(ABC):
    @abstractmethod
    def visitar_livro(self, livro): pass

    @abstractmethod
    def visitar_fruta(self, fruta): pass

# 2. Elementos da Estrutura (Aceitam a visita)
class Item(ABC):
    @abstractmethod
    def aceitar(self, visitante: Visitante): pass

class Livro(Item):
    def __init__(self, preco): self.preco = preco
    def aceitar(self, visitante):
        # O pulo do gato: o elemento chama o método específico do visitante
        visitante.visitar_livro(self)

class Fruta(Item):
    def __init__(self, peso, preco_kg):
        self.peso = peso
        self.preco_kg = preco_kg
    def aceitar(self, visitante):
        visitante.visitar_fruta(self)

# 3. Visitantes Concretos (As operações novas)
class CalculadorImposto(Visitante):
    def visitar_livro(self, livro):
        print(f"Imposto Livro: R$ {livro.preco * 0.05}")

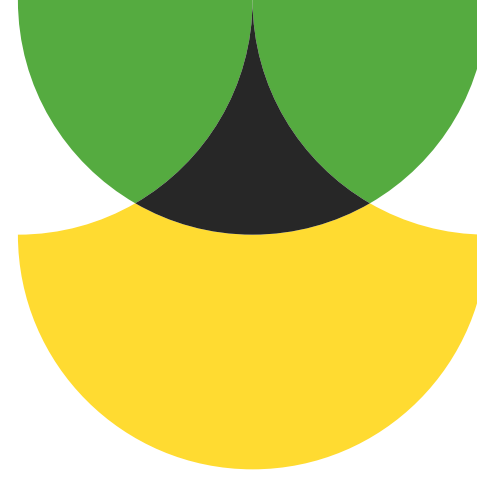
    def visitar_fruta(self, fruta):
        print(f"Imposto Fruta: R$ {(fruta.peso * fruta.preco_kg) * 0.02}")

# --- Uso Prático ---
carrinho = [Livro(50), Fruta(2, 5), Livro(100)]
perito_impuestos = CalculadorImposto()

for item in carrinho:
    item.aceitar(perito_impuestos)
```

# Vantagens:

- O objeto visitante pode acumular informações úteis.
- Obedece o princípio aberto/fechado.
- Obedece o princípio de responsabilidade única.

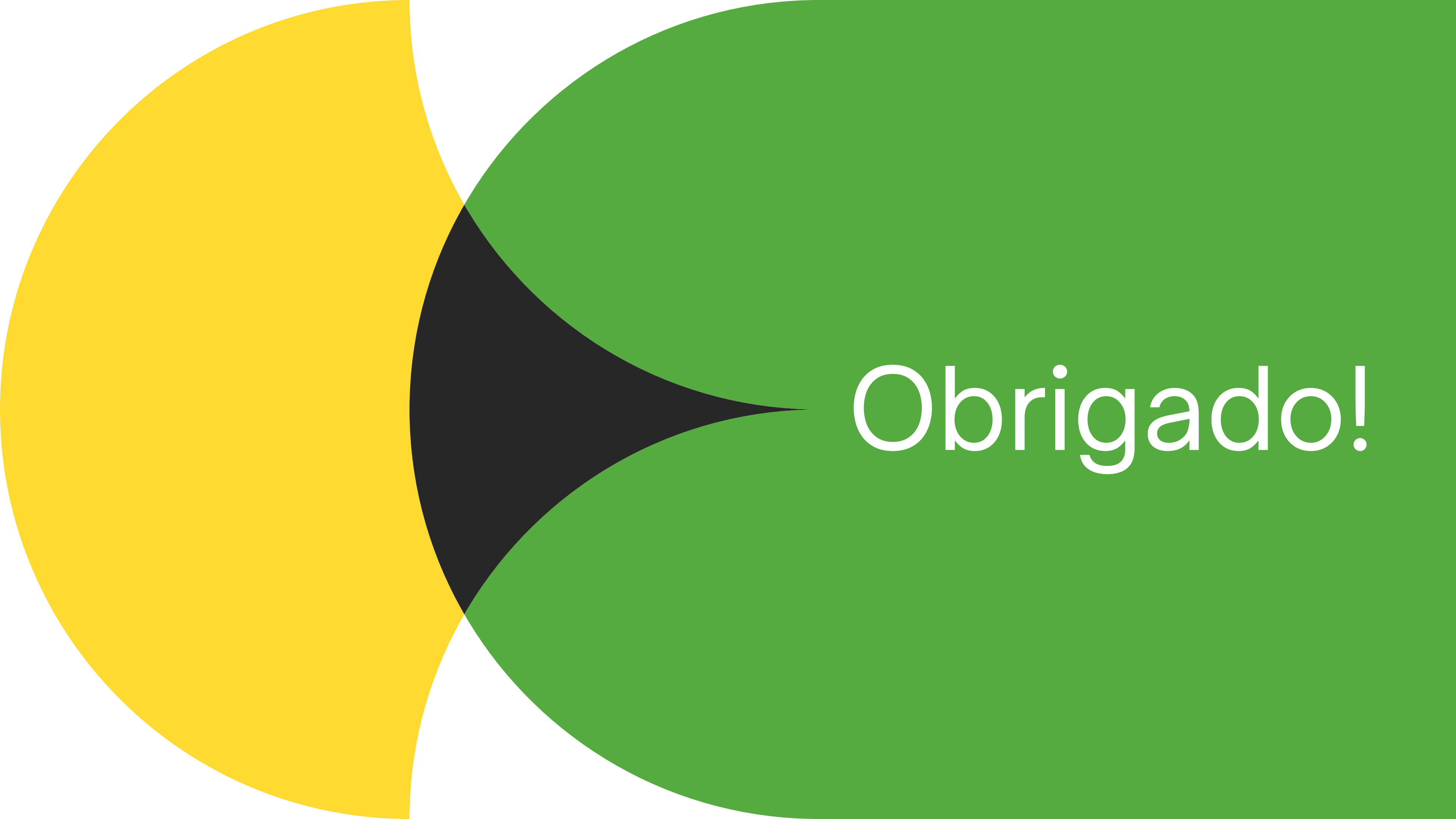


# Desvantagens:

- É necessário atualizar todos os visitantes sempre que uma classe é removida ou adicionada.
- Visitantes podem ter acesso negado a atributos e métodos privados.





A graphic featuring two large, overlapping circles. The circle on the left is yellow, and the circle on the right is green. The intersection of the two circles is filled with a dark grey or black color. The word "Obrigado!" is written in white, sans-serif font across the green circle, partially overlapping the black intersection.

Obrigado!